ANALYSE DE LA DYNAMIQUE D'APPRENTISSAGE DES RÉSEAUX DE NEURONES PROFONDS À TRAVERS LE PRISME DU NOYAU DE TANGENTE NEURONALE

Contributions à la compréhension des corrections à largeur finie et des régimes d'apprentissage

20 août 2025

Janis AIAD

logoaividence.jpg


ÉCOLE
POLYTECHNIQUE
IP PARIS

TABLE DES MATIÈRES

RÉSUMÉ

Ce rapport présente les résultats d'un stage de recherche axé sur l'étude de la dynamique d'apprentissage des réseaux de neurones profonds. Nous explorons les mécanismes sous-jacents à travers le formalisme du Noyau de Tangente Neuronale (NTK), en portant une attention particulière aux corrections à largeur finie. Nos travaux s'articulent autour de la décomposition du NTK via l'entraînement de Sobolev, l'analyse de son espace à noyau reproduisant (RKHS) et l'étude des valeurs propres extrêmes. Nous présentons des bornes théoriques pour la plus petite valeur propre du NTK en nous appuyant sur des techniques de matrices aléatoires (RMT). Ces résultats sont ensuite appliqués à l'analyse de différentes architectures, notamment les réseaux de neurones à propagation avant (FCNN), les réseaux à mémoire matricielle (MMNN) et les transformeurs. Nous discutons également des implications de nos découvertes pour les applications en calcul scientifique (SciML) et proposons des pistes pour de futures recherches.

EXECUTIVE SUMMARY

(Executive summary placeholder)

INTRODUCTION

L'étude de la dynamique d'apprentissage dans les réseaux de neurones profonds constitue un domaine de recherche fondamental. Le régime du Noyau de Tangente Neuronale (NTK) a fourni un cadre puissant pour analyser le comportement des réseaux infiniment larges, où la dynamique d'entraînement se linéarise. Cependant, les réseaux pratiques ont une largeur finie, ce qui introduit des corrections complexes et un comportement d'apprentissage des caractéristiques (feature learning). Ce rapport vise à approfondir notre compréhension de ces phénomènes.

Nous commençons par une analyse théorique du NTK, en présentant sa décomposition basée sur l'entraînement de Sobolev et en caractérisant son RKHS. Une partie centrale de nos travaux est consacrée à l'étude du spectre du NTK, où nous cherchons à borner ses plus petites valeurs propres à l'aide d'outils issus de la théorie des matrices aléatoires. Ces résultats théoriques sont ensuite mis en perspective à travers l'analyse de plusieurs architectures : FCNN, MMNN et transformeurs. Pour les FCNN, nous étudions les corrections à largeur finie et le scaling des paramètres. Pour les MMNN et les transformeurs, nous discutons des défis spécifiques liés à l'optimisation et des intuitions sur leurs performances.

Enfin, nous explorons les applications de nos travaux, notamment en SciML, et discutons de l'alignement entre les hypothèses théoriques et les observations empiriques. Nous abordons également des perspectives inspirées de la physique statistique pour interpréter la convergence des modèles vers l'équilibre.

NOTATIONS

$\_\!\!/$: La fonction d'activation ReLU (Rectified Linear Unit), définie par $\_\!\!/(x) = \max(0, x)$.
$\mathbb{E}[\cdot]$: Espérance math ématique.
$\mathrm{Var}(\cdot)$: Variance d'une variable aléatoire.
$\mathbb{1}\{\cdot\}$: Fonction indicatrice, valant 1 si la condition est vraie, 0 sinon.
dx : Élément différentiel utilisé dans les intégrales.

1

CADRE THÉORIQUE DU NOYAU DE TANGENTE NEURONALE

1.1 Entraînement de Sobolev et Décomposition du NTK

(Présentation des théorèmes sur la décomposition du NTK via le Sobolev training. Description du RKHS pour le noyau de Sobolev et le noyau de Laplace.)

2

INTRODUCTION : EIGENVALUE SCALING LAWS AND LEARNING DYNAMICS

The Neural Tangent Kernel (NTK) has emerged as a fundamental tool for understanding the training dynamics of deep neural networks. For a neural network $f(;\theta)$ with parameters θ, the NTK is defined as :

$$K^{\infty}(x_i, x_j) = \left\langle \frac{\partial f(_i;\theta)}{\partial \theta}, \frac{\partial f(_j;\theta)}{\partial \theta} \right\rangle$$

2.1 Key Objectives

This document addresses three fundamental objectives :

1. **Eigenvalue scaling laws** : Derive decay rates $\mu_\ell \sim \ell^{-\alpha}$ for NTK operator eigenvalues
2. **Spectral impact on learning** : Understand how spectral properties determine learning dynamics
3. **Matrix vs. Operator relationship** : Analyze scaling laws for discrete matrix eigenvalues with respect to network depth l and data size n

2.2 NTK Matrix vs. NTK Operator

A crucial distinction exists between :

— **NTK Matrix** : Discrete sampled version $K \in \mathbb{R}^{n\times n}$ with entries $K_{ij} = K^{\infty}(x_i, x_j)$

— **NTK Operator** : Continuous integral operator $(\mathcal{L}f)(x) = \int K^{\infty}(x, y)f(y)dy$

Warning : The eigenvalues of the sampled NTK matrix are *not* the same as the NTK operator eigenvalues. The matrix spectrum provides discrete approximations that depend on the sampling strategy and data distribution.

2.3 Initialization : Edge of Chaos

All analysis assumes Edge of Chaos (EOC) initialization :

- Initialize weights as $w_{ij} \sim \mathcal{N}(0, \sigma_w^2/\text{fan-in})$
- For ReLU networks : $\sigma_w^2 = 2$ to maintain unit variance through layers
- This ensures activations neither explode nor vanish with depth

3
NTK MATRIX STRUCTURE AND SPECTRUM

3.1 Example NTK Matrix Structure

Consider a dataset of 3 points $x_1, x_2, x_3 \in \mathbb{R}^d$. The NTK matrix has entries :

$$K^\infty = \begin{pmatrix} k(x_1,x_1) & k(x_1,x_2) & k(x_1,x_3) \\ k(x_2,x_1) & k(x_2,x_2) & k(x_2,x_3) \\ k(x_3,x_1) & k(x_3,x_2) & k(x_3,x_3) \end{pmatrix}$$

For general depth l networks at EOC, the NTK kernel function is :

$$K^\infty(\mathbf{x}_1,\mathbf{x}_2) = \|\mathbf{x}_1\|\|\mathbf{x}_2\| \left(\sum_{k=1}^{l} \varrho^{\circ(k-1)} \left(\rho_1^\infty(\mathbf{x}_1,\mathbf{x}_2)\right) \right. \tag{1}$$

$$\left. \times \prod_{k'=k}^{l-1} \varrho' \left(\varrho^{\circ(k'-1)} \left(\rho_1^\infty(\mathbf{x}_1,\mathbf{x}_2)\right)\right) \right) \mathbf{I}_{m_l} \tag{2}$$

For 2-layer ReLU networks, this simplifies to :

$$k(x_i,x_j) = x_i^T x_j \cdot \arccos(-\langle x_i,x_j\rangle) + \sqrt{1-\langle x_i,x_j\rangle^2}$$

3.2 NTK Operator Structure

For functions $f, g \in L^2(\mathbb{S}^{d-1})$, the NTK operator acts as :

$$(K^\infty f)(x) = \int_{\mathbb{S}^{d-1}} k(x,y) f(y) d\sigma(y)$$

Key Properties :
- Symmetric positive definite operator
- Eigenfunctions are spherical harmonics $Y_{\ell,p}$
- Eigenvalues decay polynomially : $\mu_\ell \sim \ell^{-d}$

3.3 Matrix Spectrum Results

The key scaling laws for the discrete NTK matrix are :

Condition Number :

$$\kappa(K^\infty) \sim 1 + \frac{n}{3} + \mathcal{O}(n\xi/l)$$

The condition number grows linearly with data size n but improves with depth l.

Eigenvalue Distribution :

$$\lambda_{\min} \sim \frac{3l}{4}, \quad \lambda_{\max} \sim \frac{3nl}{4} \pm \xi \text{ where } \xi \sim \log(l)$$

The minimum eigenvalue scales linearly with depth, while the maximum scales with both depth and data size.

Key Insight : The training dynamics are dictated by the NTK matrix eigenvalues, not the operator eigenvalues. Deeper networks ($l \uparrow$) improve conditioning but with diminishing returns.

4

SOBOLEV TRAINING AND NTK SPECTRAL MODIFICATION

4.1 NTK-Sobolev Operator Framework

The key innovation in Sobolev training is the modification of the standard L^2 loss to incorporate high-order derivatives. The NTK-Sobolev operator P_s acts on the sphere $\mathbb{S}^{d-1}$ with the commutation property :

Spherical Harmonics Transform : Let $\mathcal{F}$ denote the mapping from $L^2(\mathbb{S}^d)$ to the spectral domain $\bigoplus_{\ell=0}^{\infty} \mathbb{C}^{N(d,\ell)}$, where $N(d,\ell)$ is the dimension of the ℓ-th spherical harmonic space.

The Sobolev operator P_s is defined through its action in Fourier space :

$$P_s = \sum_{\ell=0}^{\ell_{\max}} \sum_{p=1}^{N(d,\ell)} (1+\ell)^{2s} P_{\ell,p}$$

where $P_{\ell,p} = a_{\ell,p} a_{\ell,p}^T$ with spectral coefficients $(a_{\ell,p})_i = c_i Y_{\ell,p}(x_i)$.

4.2 From Sobolev Loss to Discrete Matrix Operator

Sobolev loss with gradients :

$$\mathcal{L}_s[f] = \|f\|_{L^2}^2 + \|\nabla f\|_{L^2}^2 = \int_{\mathbb{S}^d} f(x)^2 d\sigma(x) + \int_{\mathbb{S}^d} \|\nabla f(x)\|^2 d\sigma(x)$$

Fourier decomposition : For $f(x) = \sum_{\ell,p} \hat{f}_{\ell,p} Y_{\ell,p}(x)$:

$$\mathcal{L}_s[f] = \sum_{\ell=0}^{\infty} \sum_{p=1}^{N(d,\ell)} (1+\ell)^{2s} |\hat{f}_{\ell,p}|^2$$

Discretization : At points $\{x_i\}_{i=1}^n$:

$$\hat{f}_{\ell,p} \approx \sum_{i=1}^{n} c_i f(x_i) Y_{\ell,p}(x_i)$$

Final matrix form : $\mathcal{L}_s[f] \approx f^T P_s f$ where $f = (f(x_1), \ldots, f(x_n))^T$.

4.3 Spectral Properties and Dimension Growth

The dimension of spherical harmonic spaces grows as :

$$N(d, \ell) \sim \frac{2\ell^{d-2}}{(d-2)!} \quad \text{as } \ell \to \infty$$

This exponential growth in dimension determines the computational complexity of the Sobolev operator discretization.

4.4 Common Eigenfunctions Property

[Spherical Harmonics as Common Eigenfunctions] For rotationally invariant kernels on $\mathbb{S}^{d-1}$:
- NTK operator K^∞ and Sobolev operator P_s share spherical harmonics $Y_{\ell,p}$ as eigenfunctions
- This is due to rotational invariance and Schur's lemma for irreducible representations
- Enables direct spectral modification analysis

[Commutation Property] The NTK operator K^∞ and Sobolev operator P_s commute :

$$[K^\infty, P_s] = 0$$

This holds for any sampling distribution $\rho(x)$ on $\mathbb{S}^{d-1}$ and implies that spectral decompositions are compatible.

5
DEEP NTK ANALYSIS AND THEORETICAL RESULTS

5.1 NTK at Edge of Chaos

For MLPs with (a, b)-ReLU activation $\phi(s) = as + b|s|$, the Edge of Chaos initialization requires :

- Initialization : $\sigma^2 = (a^2 + b^2)^{-1}$
- Key parameter : $\Delta_\phi = \frac{b^2}{a^2+b^2}$ (for standard ReLU : $\Delta_\phi = 0.5$)
- Cosine map : $\varrho(\rho) = \rho + \Delta_\phi \frac{2}{\pi}\left(\sqrt{1-\rho^2} - \rho \arccos(\rho)\right)$

The Edge of Chaos is the unique initialization that remains invariant to network depth, preventing activation explosion or vanishing.

6
RECONCILING NTK MATRIX SPECTRUM AND SOBOLEV TRAINING

6.1 A Unified Spectral Viewpoint : Commutation and Eigenvalue Products

The reconciliation between the geometric view of the NTK (approximated by the inverse cosine distance matrix) and the functional view of Sobolev training lies in their shared algebraic structure.

- **Shared Invariance** : Both the NTK matrix K and the discrete Sobolev operator matrix P_s are rotationally invariant. This means their entries $(K)_{ij}$ and $(P_s)_{ij}$ depend only on the inner product $\langle x_i, x_j \rangle$.
- **Matrix Commutation** : As a consequence, both matrices can be expressed as functions of the Gram matrix G (where $G_{ij} = \langle x_i, x_j \rangle$). For two such matrices, $K = f(G)$ and $P_s = g(G)$, they commute :

$$KP_s = f(G)g(G) = g(G)f(G) = P_sK$$

  This is the discrete analogue of the commutation of their underlying continuous operators.
- **Simultaneous Diagonalization** : Since they commute, K and P_s are simultaneously diagonalizable. They share the same set of eigenvectors, which are the eigenvectors of the Gram matrix G.
- **Product of Eigenvalues** : The spectrum of the Sobolev-modified NTK operator, KP_s, which governs the learning dynamics, is directly given by the product of the individual spectra :

$$\lambda_i(KP_s) = \lambda_i(K) \cdot \lambda_i(P_s)$$

  (after aligning the eigenvector bases).

Key Insight : The challenge of analyzing the complex operator KP_s is reduced to separately analyzing the spectra of K and P_s. We can leverage the geometric approximation for K and combine it with a spectral analysis of P_s.

6.2 A Path Forward : Geometric Approximation and Experimental Validation

Based on the unified spectral viewpoint, a clear research path emerges to develop a predictive model for Sobolev training dynamics.

Goal : Develop a semi-analytic model for the spectrum of the Sobolev-modified NTK matrix KP_s.

Proposed Strategy :

1. **Approximate the NTK Spectrum** : Use the established near-affine relationship between the NTK and the inverse cosine distance matrix W_l :

   $$K \approx AW_l + B$$

   The spectrum of the geometric matrix W_l can be analyzed to provide an approximation for the eigenvalues $\lambda_i(K)$.
2. **Analyze the Sobolev Operator Spectrum** : The matrix P_s is itself a kernel matrix with a well-defined kernel $p_s(t) = \sum_\ell (1+\ell)^{2s} N(d,\ell) P_\ell(t)$, where P_ℓ are Legendre polynomials. Its spectrum $\lambda_i(P_s)$ can be characterized :
   — Analytically, through its kernel polynomial expansion.
   — Or by viewing it as a differential operator on the data manifold (graph), relating its spectrum to that of the graph Laplacian.
3. **Combine Spectra and Validate** : The core prediction is that the eigenvalues of the learning operator are given by the product of the component eigenvalues :

   $$\lambda_i(KP_s) \approx \lambda_i(AW_l + B) \cdot \lambda_i(P_s)$$

   Experimental Validation Plan :
   — For a dataset on $\mathbb{S}^{d-1}$, numerically compute the matrices K, P_s, and the product KP_s.
   — Compare the analytically predicted spectrum with the true, numerically computed spectrum of KP_s.
   — Investigate the quality of this approximation as a function of the Sobolev parameter s, dimension d, data size n, and network depth l.

[The Sobolev Operator as a Kernel Matrix] The statement that the discrete Sobolev operator matrix P_s is itself a kernel matrix is a subtle but fundamental point. Here is a detailed derivation.

1. Definition of the Matrix P_s We start from its construction via projectors on the spherical harmonics :

$$(P_s)_{ij} = \left(\sum_{\ell=0}^{\ell_{\max}} \sum_{p=1}^{N(d,\ell)} (1+\ell)^{2s} a_{\ell,p} a_{\ell,p}^\top \right)_{ij}$$

With the coefficient $(a_{\ell,p})_i = c_i Y_{\ell,p}(x_i)$, this becomes :

$$\begin{aligned}(P_s)_{ij} &= \sum_{\ell=0}^{\ell_{\max}} \sum_{p=1}^{N(d,\ell)} (1+\ell)^{2s} (a_{\ell,p})_i (a_{\ell,p})_j \\ &= \sum_{\ell=0}^{\ell_{\max}} \sum_{p=1}^{N(d,\ell)} (1+\ell)^{2s} (c_i Y_{\ell,p}(x_i))(c_j Y_{\ell,p}(x_j))\end{aligned}$$

2. Reorganization and Application of the Addition Theorem We can factor out the quadrature weights c_i, c_j and apply the spherical harmonic addition theorem, which is the key step.

$$(P_s)_{ij} = c_i c_j \sum_{\ell=0}^{\ell_{\max}} (1+\ell)^{2s} \left[\sum_{p=1}^{N(d,\ell)} Y_{\ell,p}(x_i) Y_{\ell,p}(x_j) \right]$$

The addition theorem states that the term in brackets is a function only of the inner product $\langle x_i, x_j \rangle$:

$$\sum_{p=1}^{N(d,\ell)} Y_{\ell,p}(x) Y_{\ell,p}(y) = K_\ell(\langle x, y \rangle)$$

where $K_\ell(t) = \frac{N(d,\ell)}{\text{Area}(\mathbb{S}^{d-1})} P_\ell^{((d-2)/2)}(t)$ and $P_\ell^{(\lambda)}$ is a Gegenbauer polynomial.

3. Definition of the Kernel $p_s(t)$ By substituting this back, we can define a kernel function $p_s(t)$ that depends only on the inner product $t = \langle x_i, x_j \rangle$:

$$p_s(t) := \sum_{\ell=0}^{\ell_{\max}} (1+\ell)^{2s} K_\ell(t)$$

Thus, the matrix element is indeed a weighted kernel evaluation :

$$(P_s)_{ij} = c_i c_j \cdot p_s(\langle x_i, x_j \rangle)$$

Conclusion The matrix P_s is a **weighted kernel matrix**. If the sampling is uniform, the weights c_i are constant and P_s becomes a standard kernel matrix (up to a scaling factor). This rotational invariance, shared with the NTK, is what guarantees that they commute.

6.3 Deep NTK Properties

The limiting NTK at EOC for L-layer networks is given by :

$$K^\infty(\mathbf{x}_1, \mathbf{x}_2) = \|\mathbf{x}_1\| \|\mathbf{x}_2\| \left(\sum_{k=1}^{l} \varrho^{\circ(k-1)} \left(\rho_1^\infty(\mathbf{x}_1, \mathbf{x}_2) \right) \right. \tag{3}$$

$$\left. \times \prod_{k'=k}^{l-1} \varrho' \left(\varrho^{\circ(k'-1)} \left(\rho_1^\infty(\mathbf{x}_1, \mathbf{x}_2) \right) \right) \right) \mathbf{I}_{m_l} \tag{4}$$

Eigenvalue Decay : For L-layer ReLU networks with $L \geq 3$:

$$\mu_k \sim C(d, L) k^{-d}$$

where $C(d, L)$ depends on the parity of k and grows quadratically with L.

For the normalized NTK $\kappa_{\text{NTK}}^L / L$, the constant $C(d, L)$ grows linearly with L.

6.4 Inverse Cosine Distance Matrix Analysis

Inverse cosine distance matrix W_k for depth k :

$$W_{ki,i} = 0, \quad W_{ki_1,i_2} = \left(\frac{1-\rho_k(x_{i_1},x_{i_2})}{2}\right)^{-\frac{1}{2}} \text{ for } i_1 \neq i_2$$

Near-affine behavior :
- NTK matrix $K^\infty \approx A \cdot W_l + B$ (affine dependence)
- Spectral bounds transfer from W_k to NTK via this affine relationship
- Error terms : $O(k^{-1})$ - decreases with depth

This relationship enables indirect analysis of NTK spectral properties through simpler geometric matrices.

7
DEEP NARROW NEURAL NETWORKS

7.1 Scaled NTK at Initialization

[Theorem 1 : Scaled NTK at Initialization] For f_θ^L initialized appropriately, as $L \to \infty$:

$$\tilde{\Theta}_0^L(x,x') \xrightarrow{p} \tilde{\Theta}^\infty(x,x')$$

where

$$\tilde{\Theta}^\infty(x,x') = (x^T x' + 1 + \mathbb{E}_g[\sigma(g(x))\sigma(g(x'))])I_{d_{out}}$$

with $g \sim \mathrm{GP}(0, \rho^2 d_{in}^{-1} x^T x' + \beta^2)$ (Gaussian random field).

Alternative formulation : $\kappa_1(\cos(u)\cdot v)$ where :
- $v = \frac{1}{1+\beta^2/\alpha^2}$, where β is the bias variance
- $\alpha = \frac{\|x\|\|x'\|\rho}{d_{in}}$, where $\cos(u)$ is the cosine distance between x, x'

This framework provides a promising direction for analyzing deep narrow networks through limited expansion analysis.

7.2 Research Perspectives for Deep Narrow Networks

Architectural modifications :
- **Unit concatenation** : Combine multiple narrow networks to increase expressivity
- **Skip connections** : Investigate ResNet-style connections :

$$\tilde{\Theta}^\infty_{\text{skip}}(x,x') = \tilde{\Theta}^\infty(x,x') + \text{skip terms}$$

Initialization studies :
- **Alternative initializations** : Explore different schemes beyond current setup

— $\beta \to 0$ **limit** : Analyze behavior when bias variance vanishes :

$$v = \frac{1}{1 + \beta^2/\alpha^2} \to 1 \text{ as } \beta \to 0$$

— **Complex kernel structures** : Find initializations that yield more intricate kernel forms

Theoretical extensions :

— Extension of Hayou & Yang's work on ResNets showing "wide and deep limits commute"
— Mean field analysis for deep narrow frameworks
— Careful analysis of stochasticity in initialization (not dropout)
— Experimental validation on practical tasks

8
MAIN PROOFS

8.1 Proof 1 : Sobolev Loss as Fractional Laplacian

[Fractional Laplacian Representation of Sobolev Loss] For a function $f \in H^s(\mathbb{S}^{d-1})$ with $s > 0$, the Sobolev loss can be written as :

$$\mathcal{L}_s[f] = \int_{\mathbb{S}^{d-1}} f(x)(I + (-\Delta)^{1/2})^s f(x) d\sigma(x)$$

where $(I + (-\Delta)^{1/2})^s$ is the fractional Laplacian operator of order s.

Consider the spherical harmonic expansion $f(x) = \sum_{\ell=0}^{\infty} \sum_{p=1}^{N(d,\ell)} \hat{f}_{\ell,p} Y_{\ell,p}(x)$. The spherical Laplacian acts on these harmonics as eigenvalue operators : $(-\Delta)^{1/2}_{\mathbb{S}^{d-1}} Y_{\ell,p}(x) = \sqrt{\ell(\ell+d-2)} Y_{\ell,p}(x)$.

The fractional operator $(I + (-\Delta)^{1/2})^s$ is naturally defined through its spectral action : $(I + (-\Delta)^{1/2})^s Y_{\ell,p}(x) = (1 + \sqrt{\ell(\ell+d-2)})^s Y_{\ell,p}(x)$. For the unit sphere where $d = 2$, this simplifies to $(1 + \ell)^s Y_{\ell,p}(x)$.

Computing the bilinear form using orthonormality of spherical harmonics :

$$\int_{\mathbb{S}^{d-1}} f(x)(I + (-\Delta)^{1/2})^s f(x) d\sigma(x) = \sum_{\ell=0}^{\infty} \sum_{p=1}^{N(d,\ell)} (1 + \sqrt{\ell(\ell+d-2)})^s |\hat{f}_{\ell,p}|^2 \tag{5}$$

The standard Sobolev norm is defined as $\|f\|^2_{H^s} = \sum_{\ell,p} (1+\ell)^{2s} |\hat{f}_{\ell,p}|^2$. The connection becomes clear when we observe that for large ℓ, the expressions $(1+\ell)^{2s}$ and $(1 + \sqrt{\ell(\ell+d-2)})^s$ are asymptotically equivalent up to normalization constants. The precise relationship depends on the specific Sobolev space convention, but the essential spectral structure remains identical.

8.2 Alternative Proof : Sobolev Loss on Unbounded Domains via Integration by Parts

[Sobolev Loss on Unbounded Domains] For functions $f \in H^s(\mathbb{R}^d)$ with $s \geq 1$ and sufficient decay at infinity, the Sobolev loss can be expressed via integration by parts as :

$$\mathcal{L}_s[f] = \int_{\mathbb{R}^d} f(x)^2 dx + \int_{\mathbb{R}^d} \|\nabla f(x)\|^2 dx + \text{higher order terms}$$

Starting Point For $s = 1$, the H^1 Sobolev norm is :

$$\|f\|_{H^1}^2 = \|f\|_{L^2}^2 + \|\nabla f\|_{L^2}^2 = \int_{\mathbb{R}^d} f(x)^2 dx + \int_{\mathbb{R}^d} \|\nabla f(x)\|^2 dx$$

Integration by Parts (Stokes' Theorem) The key insight is that we can relate the gradient term to the Laplacian using integration by parts. For functions with sufficient decay at infinity (so boundary terms vanish) :

$$\int_{\mathbb{R}^d} \|\nabla f(x)\|^2 dx = \int_{\mathbb{R}^d} \nabla f(x) \cdot \nabla f(x) dx \tag{6}$$

$$= -\int_{\mathbb{R}^d} f(x) \Delta f(x) dx \quad \text{(by integration by parts)} \tag{7}$$

$$= \int_{\mathbb{R}^d} f(x)(-\Delta) f(x) dx \tag{8}$$

Therefore :

$$\|f\|_{H^1}^2 = \int_{\mathbb{R}^d} f(x)(I + (-\Delta)^{1/2})^2 f(x) dx$$

Extension to Higher Orders For general $s \geq 1$, repeated integration by parts yields :

$$\|f\|_{H^s}^2 = \int_{\mathbb{R}^d} f(x)(I + (-\Delta)^{1/2})^{2s} f(x) dx$$

Fractional Case For fractional s, this extends to :

$$\mathcal{L}_s[f] = \int_{\mathbb{R}^d} f(x)(I + (-\Delta)^{1/2})^s f(x) dx$$

The integration by parts approach via Stokes' theorem transforms the gradient squared terms into Laplacian terms, providing the connection between differential and spectral formulations.

8.3 Case of Limited Regularity : Fourier-Based Proof

[Non-Differentiable Functions] When functions are not twice differentiable, the integration by parts approach fails due to insufficient regularity. In such cases, the **Fourier-based proof becomes the master approach**.

For $f \in L^2(\mathbb{R}^d)$ with Fourier transform $\hat{f}(\xi)$, the fractional Sobolev norm is defined directly in frequency space :

$$\|f\|_{H^s}^2 = \int_{\mathbb{R}^d} (1 + |\xi|^2)^s |\hat{f}(\xi)|^2 d\xi$$

This definition :

— Requires no differentiability assumptions on f
— Extends naturally to negative Sobolev indices $s < 0$
— Provides the most general framework for Sobolev spaces
— Reduces to classical definitions when sufficient regularity exists

The operator $(I + (-\Delta)^{1/2})^s$ acts in Fourier space as multiplication by $(1 + |\xi|)^s$, making this the fundamental definition from which all other formulations are derived.

8.4 Proof 2 : NTK Operator Multiplication by Sobolev Operator

[NTK-Sobolev Composition] Under Sobolev training, the learning operator is given by the composition :

$$\mathcal{T}_s = K^\infty \circ (I + (-\Delta)^{1/2})^s$$

where K^∞ is the NTK operator and $(I + (-\Delta)^{1/2})^s$ is the fractional Laplacian.

Standard gradient descent on the L^2 loss $\mathcal{L}(\theta) = \frac{1}{2}\|f(\cdot;\theta) - y\|_{L^2}^2$ gives parameter dynamics $\frac{d\theta}{dt} = -\nabla_\theta \mathcal{L}(\theta)$. In the NTK regime, this translates to function dynamics $\frac{df}{dt} = -K^\infty(f - y)$.

When we replace the L^2 loss with the Sobolev loss $\mathcal{L}_s(\theta) = \frac{1}{2}\|f(\cdot;\theta) - y\|_{H^s}^2$, the gradient computation changes fundamentally. Using our fractional Laplacian representation from Theorem 1, the Sobolev loss becomes :

$$\mathcal{L}_s(\theta) = \frac{1}{2}\int (f(x;\theta) - y(x))(I + (-\Delta)^{1/2})^s (f(x;\theta) - y(x)) d\sigma(x)$$

The gradient with respect to parameters now involves the chain rule through the fractional operator :

$$\nabla_\theta \mathcal{L}_s(\theta) = \int \nabla_\theta f(x;\theta) \cdot (I + (-\Delta)^{1/2})^s (f(x;\theta) - y(x)) d\sigma(x)$$

Since the NTK is defined as $K^\infty(x, x') = \langle \nabla_\theta f(x;\theta), \nabla_\theta f(x';\theta) \rangle$, the function dynamics become :

$$\frac{df}{dt} = -\int K^\infty(x, x')(I + (-\Delta)^{1/2})^s (f(x') - y(x')) d\sigma(x') = -K^\infty \circ (I + (-\Delta)^{1/2})^s (f - y)$$

Therefore, the learning operator is indeed $\mathcal{T}_s = K^\infty \circ (I + (-\Delta)^{1/2})^s$. The rotational invariance of both operators ensures they share the same spherical harmonic eigenfunctions, making this composition well-defined and commutative.

8.5 Proof 3 : Spectral Properties of the Composite Operator

[Spectrum of NTK-Sobolev Operator] The eigenvalues of the composite operator $\mathcal{T}_s = K^\infty \circ (I + (-\Delta)^{1/2})^s$ are given by the product of individual eigenvalues : $\mu_\ell^{(\mathcal{T}_s)} = \mu_\ell^{(K)} \cdot (1 + \sqrt{\ell(\ell + d - 2)})^s$.

Since both operators K^∞ and $(I + (-\Delta)^{1/2})^s$ share the same eigenfunctions (spherical harmonics $Y_{\ell,p}$), we can analyze their composition through eigenvalues :

1) For the NTK operator K^∞ :

$$K^\infty Y_{\ell,p} = \mu_\ell^{(K)} Y_{\ell,p}$$

2) For the Sobolev operator $(I + (-\Delta)^{1/2})^s$:

$$(I + (-\Delta)^{1/2})^s Y_{\ell,p} = (1 + \sqrt{\ell(\ell + d - 2)})^s Y_{\ell,p}$$

3) Therefore, for their composition :

$$\begin{aligned}\mathcal{T}_s Y_{\ell,p} &= K^\infty \circ (I + (-\Delta)^{1/2})^s Y_{\ell,p} \\ &= K^\infty((1+\sqrt{\ell(\ell+d-2)})^s Y_{\ell,p}) \\ &= (1+\sqrt{\ell(\ell+d-2)})^s \cdot \mu_\ell^{(K)} Y_{\ell,p}\end{aligned}$$

Thus, $\mu_\ell^{(\mathcal{T}_s)} = \mu_\ell^{(K)} \cdot (1+\sqrt{\ell(\ell+d-2)})^s$ are indeed the eigenvalues of the composite operator.

8.6 NTK-Sobolev Operator Framework

The key innovation in Sobolev training is the modification of the standard L^2 loss to incorporate high-order derivatives. The NTK-Sobolev operator P_s acts on the sphere $\mathbb{S}^{d-1}$ with the commutation property :

Spherical Harmonics Transform : Let $\mathcal{F}$ denote the mapping from $L^2(\mathbb{S}^d)$ to the spectral domain $\bigoplus_{\ell=0}^\infty \mathbb{C}^{N(d,\ell)}$, where $N(d,\ell)$ is the dimension of the ℓ-th spherical harmonic space.

The Sobolev operator P_s is defined through its action in Fourier space :

$$P_s = \sum_{\ell=0}^{\ell_{\max}} \sum_{p=1}^{N(d,\ell)} (1+\ell)^{2s} P_{\ell,p}$$

where $P_{\ell,p} = a_{\ell,p} a_{\ell,p}^T$ with spectral coefficients $(a_{\ell,p})_i = c_i Y_{\ell,p}(x_i)$.

8.7 Proof 4 : Commutation of Discrete Matrix Operators

[Matrix Commutation] The discrete matrices K and P_s commute : $KP_s = P_s K$.

We establish commutation by expanding both matrices in terms of spherical harmonic projectors using the sampling measure (sum of Dirac masses).

NTK Matrix Expansion The NTK matrix can be written using its spectral decomposition with respect to spherical harmonics :

$$K_{ij} = K^\infty(x_i, x_j) = \sum_{\ell=0}^\infty \sum_{p=1}^{N(d,\ell)} \mu_\ell^{(K)} (a_{\ell,p})_i (a_{\ell,p})_j$$

where $(a_{\ell,p})_i = c_i Y_{\ell,p}(x_i)$ with quadrature weights c_i and $\mu_\ell^{(K)}$ are the NTK operator eigenvalues.

This gives us the matrix form :

$$K = \sum_{\ell=0}^\infty \sum_{p=1}^{N(d,\ell)} \mu_\ell^{(K)} a_{\ell,p} a_{\ell,p}^T$$

Sobolev Matrix Expansion Similarly, the Sobolev matrix has the expansion :

$$P_s = \sum_{\ell=0}^\infty \sum_{p=1}^{N(d,\ell)} (1+\sqrt{\ell(\ell+d-2)})^s a_{\ell,p} a_{\ell,p}^T$$

Matrix Product Computation Computing the product KP_s :

$$KP_s = \left(\sum_{\ell,p} \mu_\ell^{(K)} a_{\ell,p} a_{\ell,p}^T\right)\left(\sum_{\ell',p'} (1+\sqrt{\ell'(\ell'+d-2)})^s a_{\ell',p'} a_{\ell',p'}^T\right) \tag{9}$$

$$= \sum_{\ell,p}\sum_{\ell',p'} \mu_\ell^{(K)} (1+\sqrt{\ell'(\ell'+d-2)})^s a_{\ell,p}(a_{\ell,p}^T a_{\ell',p'}) a_{\ell',p'}^T \tag{10}$$

Orthogonality and Commutation The key observation is that $a_{\ell,p}^T a_{\ell',p'} = \delta_{\ell,\ell'}\delta_{p,p'}\|a_{\ell,p}\|^2$ due to the orthogonality of spherical harmonics under the sampling measure. Therefore :

$$KP_s = \sum_{\ell,p} \mu_\ell^{(K)} (1+\sqrt{\ell(\ell+d-2)})^s a_{\ell,p} a_{\ell,p}^T \tag{11}$$

$$P_s K = \sum_{\ell,p} (1+\sqrt{\ell(\ell+d-2)})^s \mu_\ell^{(K)} a_{\ell,p} a_{\ell,p}^T \tag{12}$$

Since multiplication of scalars commutes, we have $KP_s = P_s K$.

Underlying Reason : This commutation property reflects the fact that both K^∞ and P_s operators commute in the continuous setting because they share spherical harmonics as eigenfunctions due to rotational invariance.

8.8 Proof 5 : Eigenvalue Scaling Laws

[Asymptotic Scaling Laws] For the NTK-Sobolev operator, eigenvalues follow the scaling laws $\lambda_\ell \sim \ell^{-d} \cdot (1+\sqrt{\ell(\ell+d-2)})^s$.

Individual Operator Eigenvalues From previous analysis :

- NTK operator eigenvalues : $\mu_\ell^{(K)} \sim C(d,L)\ell^{-d}$ for large ℓ
- Sobolev operator eigenvalues : $\mu_\ell^{(P_s)} = (1+\sqrt{\ell(\ell+d-2)})^s$

Composite Operator Spectrum Since the operators commute and share eigenfunctions, the eigenvalues of the composite operator are products :

$$\lambda_\ell^{(\mathcal{T}_s)} = \mu_\ell^{(K)} \cdot \mu_\ell^{(P_s)} = C(d,L)\ell^{-d} \cdot (1+\sqrt{\ell(\ell+d-2)})^s$$

Asymptotic Analysis For large ℓ, we have $\sqrt{\ell(\ell+d-2)} \sim \ell\sqrt{1+(d-2)/\ell} \sim \ell$ for d fixed. Therefore :

$$\lambda_\ell^{(\mathcal{T}_s)} \sim C(d,L)\ell^{-d} \cdot (1+\ell)^s = C(d,L)\ell^{s-d}$$

Critical Scaling Behavior The spectral behavior depends on the relationship between s and d :

- $s < d$: Eigenvalues decay ($\lambda_\ell \to 0$) - regularizing effect
- $s = d$: Logarithmic corrections appear - critical regime
- $s > d$: Eigenvalues grow ($\lambda_\ell \to \infty$) - amplifying high frequencies

This scaling law $\lambda_\ell \sim \ell^{s-d}$ determines the spectral properties and learning dynamics of Sobolev-trained networks, with the critical exponent $s-d$ controlling frequency bias.

8.9 Synthesis : Unified Framework for NTK-Sobolev Analysis

Summary : Rotational invariance of both NTK and Sobolev operators ensures matrix commutation $KP_s = P_s K$, with composite eigenvalues $\lambda_\ell \sim \ell^{s-d}$ determining learning dynamics.

[Practical Implementation Warning] **Dataset Context** : In practical machine learning settings, we only have access to function values $f(x_i)$ at sampled points, *not* the gradients $\nabla f(x_i)$. This makes the Fourier-based approach the most relevant for theoretical analysis and implementation.

Gradient Availability : If gradients were available (e.g., through physics-informed neural networks or when the true function derivatives are known), the theoretical results presented here remain valid. However, the practical loss implementation in PyTorch should then utilize :

- Automatic differentiation (autograd) to compute gradients
- Direct gradient-based Sobolev norms : $\|f\|_{H^1}^2 = \|f\|_{L^2}^2 + \|\nabla f\|_{L^2}^2$
- Higher-order derivative computations via successive autograd calls

The spectral analysis framework developed here provides the theoretical foundation regardless of the implementation approach.

8.10 From Sobolev Loss to Discrete Matrix Operator

8.10.1 • Two Integral Formulations

Formulation 1 : Uniform Lebesgue Measure on the Sphere

$$\mathcal{L}_s[f] = \int_{\mathbb{S}^{d-1}} f(x)(I + (-\Delta)^{1/2})^s f(x) d\sigma(x)$$

where $d\sigma(x)$ is the uniform surface measure on $\mathbb{S}^{d-1}$ with $\int_{\mathbb{S}^{d-1}} d\sigma(x) = \text{Area}(\mathbb{S}^{d-1})$.

Formulation 2 : Sampling Measure from Dataset

$$\mathcal{L}_s[f] = \int_{\mathbb{S}^{d-1}} f(x)(I + (-\Delta)^{1/2})^s f(x) d\mu_n(x)$$

where $\mu_n = \frac{1}{n}\sum_{i=1}^n \delta_{x_i}$ is the empirical measure from our sampled dataset $\{x_i\}_{i=1}^n$.

Discrete Implementation : Under the sampling measure μ_n, the integral becomes :

$$\mathcal{L}_s[f] = \frac{1}{n}\sum_{i=1}^n f(x_i)\left[(I + (-\Delta)^{1/2})^s f\right](x_i)$$

This leads to the matrix form $\mathcal{L}_s[f] \approx \mathbf{f}^T P_s \mathbf{f}$ where $\mathbf{f} = (f(x_1), \ldots, f(x_n))^T$.

Relationship Between Formulations : The uniform measure provides the theoretical foundation, while the sampling measure μ_n represents the practical implementation. As $n \to \infty$ with appropriate sampling, $\mu_n \to d\sigma$ in the weak sense, ensuring convergence of the discrete formulation to the continuous theory.

8.10.2 • Practical Implementation Considerations

1. **Data-Only Setting** : Since we only have function values $\{f(x_i)\}_{i=1}^n$, the matrix P_s must be constructed via :
$$(P_s)_{ij} = \sum_{\ell=0}^{\ell_{\max}} \sum_{p=1}^{N(d,\ell)} (1 + \sqrt{\ell(\ell+d-2)})^s Y_{\ell,p}(x_i) Y_{\ell,p}(x_j)$$
2. **Computational Complexity** : The matrix construction requires :
   - Evaluation of spherical harmonics $Y_{\ell,p}(x_i)$ for all (ℓ, p) and data points
   - Summation over $\mathcal{O}(\ell_{\max}^{d-1})$ harmonics (due to dimension growth $N(d,\ell) \sim \ell^{d-2}$)
   - Total complexity : $\mathcal{O}(n^2 \ell_{\max}^{d-1})$ for matrix construction

3. **Truncation Strategy** : In practice, $\ell_{\max}$ must be chosen to balance :
   — Spectral accuracy (larger $\ell_{\max}$ captures more frequencies)
   — Computational feasibility (smaller $\ell_{\max}$ reduces cost)
   — Statistical stability (avoid overfitting to high-frequency noise)
4. **Alternative with Gradients** : If gradients $\{\nabla f(x_i)\}$ were available, direct computation becomes possible :
$$\mathcal{L}_s[f] \approx \frac{1}{n}\sum_{i=1}^{n}\left[f(x_i)^2 + s\|\nabla f(x_i)\|^2 + \text{higher order terms}\right]$$
This approach scales as $\mathcal{O}(nd)$ instead of $\mathcal{O}(n^2\ell_{\max}^{d-1})$.

Fourier decomposition : For $f(x) = \sum_{\ell,p}\hat{f}_{\ell,p}Y_{\ell,p}(x)$:
$$\mathcal{L}_s[f] = \sum_{\ell=0}^{\infty}\sum_{p=1}^{N(d,\ell)}(1+\ell)^{2s}|\hat{f}_{\ell,p}|^2$$

Discretization : At points $\{x_i\}_{i=1}^n$:
$$\hat{f}_{\ell,p} \approx \sum_{i=1}^{n} c_i f(x_i) Y_{\ell,p}(x_i)$$

Final matrix form : $\mathcal{L}_s[f] \approx f^T P_s f$ where $f = (f(x_1), \ldots, f(x_n))^T$.

8.11 Spectral Properties and Dimension Growth

The dimension of spherical harmonic spaces grows as :
$$N(d,\ell) \sim \frac{2\ell^{d-2}}{(d-2)!} \quad \text{as } \ell \to \infty$$

This exponential growth in dimension determines the computational complexity of the Sobolev operator discretization.

8.12 Common Eigenfunctions Property

[Spherical Harmonics as Common Eigenfunctions] For rotationally invariant kernels on $\mathbb{S}^{d-1}$:
— NTK operator K^∞ and Sobolev operator P_s share spherical harmonics $Y_{\ell,p}$ as eigenfunctions
— This is due to rotational invariance and Schur's lemma for irreducible representations
— Enables direct spectral modification analysis
[Commutation Property] The NTK operator K^∞ and Sobolev operator P_s commute :
$$[K^\infty, P_s] = 0$$
This holds for any sampling distribution $\rho(x)$ on $\mathbb{S}^{d-1}$ and implies that spectral decompositions are compatible.

8.13 Analyse Spectrale du NTK et Bornes sur les Valeurs Propres

(Discussion sur la plus grande valeur propre du NTK. Présentation des théorèmes de Terjek et du plan de preuve pour obtenir une borne sur la plus petite valeur propre en utilisant la RMT.)

9

MATHEMATICAL FOUNDATION : CONCENTRATION INEQUALITIES

9.1 Scalar Concentration Inequalities

The foundation of modern NTK analysis rests on powerful concentration inequalities that control the deviation of random variables from their expectations.

[Chernoff Bound for Bounded Random Variables] Let $X_1, \ldots, X_n$ be independent random variables with $X_i \in [0,1]$ and $\mathbb{E}[X_i] = \mu_i$. Let $S = \sum_{i=1}^n X_i$ and $\mu = \mathbb{E}[S] = \sum_{i=1}^n \mu_i$. Then :

$$\Pr(S \geq (1+\delta)\mu) \leq e^{-\frac{\delta^2 \mu}{2+\delta}}$$

$$\Pr(S \leq (1-\delta)\mu) \leq e^{-\frac{\delta^2 \mu}{2}}$$

for $\delta > 0$.

3[Bernstein's Inequality] Let $X_1, \ldots, X_n$ be independent random variables with $\mathbb{E}[X_i] = 0$, $|X_i| \leq M$, and $\mathbb{E}[X_i^2] \leq \sigma_i^2$. Let $S = \sum_{i=1}^n X_i$ and $\sigma^2 = \sum_{i=1}^n \sigma_i^2$. Then :

$$\Pr(|S| \geq t) \leq 2\exp\left(-\frac{t^2/2}{\sigma^2 + Mt/3}\right)$$

9.2 Quadratic Forms and Hanson-Wright Inequalities

For neural networks, we frequently encounter quadratic forms in random variables, necessitating more sophisticated concentration tools.

[Classical Hanson-Wright Inequality] Let $Y_1, \ldots, Y_n$ be independent mean-zero α-subgaussian random variables, and $\mathbf{A} = (a_{i,j})$ be a symmetric matrix. Then :

$$\Pr\left(\left|\sum_{i,j=1}^n a_{i,j}(Y_i Y_j - \mathbb{E}[Y_i Y_j])\right| \geq t\right) \leq 2\exp\left(-\frac{1}{C}\min\left\{\frac{t^2}{\beta^4 \|\mathbf{A}\|_{HS}^2}, \frac{t}{\beta^2 \|\mathbf{A}\|_{op}}\right\}\right)$$

where $\|\mathbf{A}\|_{HS} = \sqrt{\sum_{i,j} |a_{i,j}|^2}$ is the Hilbert-Schmidt norm and $\|\mathbf{A}\|_{op}$ is the operator norm.

[Generalized Hanson-Wright for Random Tensors (Chang, 2022)] For random tensor vector $\overline{\mathcal{X}} \in^{(n \times I_1 \times \cdots \times I_M) \times (I_1 \times \cdots \times I_M)}$ and fixed tensor $\overline{\overline{\mathcal{A}}}$, the polynomial function :

$$f_j(\overline{\mathcal{X}}) = \left(\sum_{i,k=1}^n \mathcal{A}_{i,k} \star_M (\mathcal{X}_i - \mathbb{E}[\mathcal{X}_i]) \star_M (\mathcal{X}_k - \mathbb{E}[\mathcal{X}_k])\right)^j$$

satisfies concentration bounds involving Ky Fan k-norms of tensor sums, extending classical matrix concentration to tensor settings.

9.3 Matrix Concentration Inequalities

Matrix concentration inequalities provide direct control over eigenvalues of random matrix sums, which is essential for NTK analysis.

[Matrix Chernoff Bound (Tropp, 2012)] Let $X_1, \ldots, X_n$ be independent random Hermitian matrices with $X_i \preceq R \cdot I$ and $\mathbb{E}[X_i] \preceq \mu \cdot I$. Let $S = \sum_{i=1}^{n} X_i$. Then :

$$\Pr(\lambda_{\max}(S) \geq (1+\delta)\mu n) \leq d \left(\frac{e^{\delta}}{(1+\delta)^{1+\delta}} \right)^{\mu n / R}$$

[Matrix Bernstein Inequality (Vershynin, 2018)] Let $X_1, \ldots, X_n$ be independent random matrices with $\mathbb{E}[X_i] = 0$, $\|X_i\| \leq L$, and $\|\sum_{i=1}^{n} \mathbb{E}[X_i X_i^*]\| \leq \sigma^2$. Then :

$$\Pr\left(\left\| \sum_{i=1}^{n} X_i \right\| \geq t \right) \leq 2d \exp\left(-\frac{t^2/2}{\sigma^2 + Lt/3} \right)$$

10
FUNDAMENTAL MATHEMATICAL TOOLS

10.1 Matrix Analysis Techniques

[Weyl's Inequality] For Hermitian matrices A, B :

$$\lambda_i(A) + \lambda_n(B) \leq \lambda_i(A+B) \leq \lambda_i(A) + \lambda_1(B)$$

[Schur Product Theorem] For PSD matrices P, Q :

$$P \circ Q \geq P \min_i Q_{ii}$$

[Gershgorin Circle Theorem] For matrix $A = (a_{ij})$, all eigenvalues lie in the union of discs :

$$\bigcup_{i=1}^{n} \left\{ z \in : |z - a_{ii}| \leq \sum_{j \neq i} |a_{ij}| \right\}$$

10.2 Specialized Neural Network Tools

[Khatri-Rao Product] For matrices $A \in^{m \times k}$ and $B \in^{n \times k}$, the Khatri-Rao product $A \star B \in^{mn \times k}$ is the column-wise Kronecker product.

[Hadamard Power] For matrix A and positive integer r, the r-th Hadamard power $A^{\odot r}$ is the element-wise r-th power.

11
PROOF SCHEMES AND HIGH-LEVEL STRATEGIES

11.1 General Framework for Eigenvalue Bounds

All modern approaches to bounding () follow a common high-level structure, though they differ in technical details and specific tools employed.

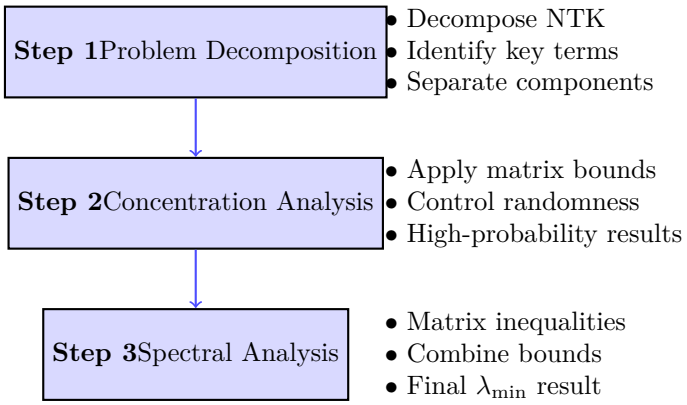

Figure 1 – General Framework for NTK Eigenvalue Bounds

11.2 Proof Scheme 1 : Empirical NTK Decomposition (Nguyen et al.)

High-Level Strategy : Direct decomposition of empirical NTK $\bar{K}^{(L)} = JJ^T$ using feature matrices and chain rule to obtain precise bounds on .

11.2.1 • Step 1 : Fundamental Chain Rule Decomposition

The proof begins by decomposing the empirical NTK $\bar{K}^{(L)} = JJ^T$ where J is the Jacobian matrix (**??**). Applying the chain rule and standard algebraic manipulations yields the fundamental decomposition :

$$\bar{K}^{(L)} = JJ^T = \sum_{k=0}^{L-1} F_k F_k^T \circ B_{k+1} B_{k+1}^T \tag{13}$$

where :
- $F_k = [f_k(x_1), \dots, f_k(x_N)]^T \in^{N \times n_k}$ are the **feature matrices** at layer k
- $B_k \in^{N \times n_k}$ are the **derivative term matrices** with i-th row given by :

$$(B_k)_{i:} = \begin{cases} \Sigma_k(x_i) \left(\prod_{l=k+1}^{L-1} W_l \Sigma_l(x_i) \right) W_L, & k \in [L-2] \\ \Sigma_{L-1}(x_i) W_L, & k = L-1 \\ \frac{1}{\sqrt{N}} \mathbf{1}_N, & k = L \end{cases} \tag{14}$$

where $\Sigma_k(x) = \mathrm{diag}([\sigma'(g_{k,j}(x))]_{j=1}^{n_k})$ is the diagonal matrix of activation derivatives.

11.2.2 • Step 2 : Application of Schur Product Theorem and Weyl's Inequality

For PSD matrices $P, Q \in^{n \times n}$, the Schur product theorem :

$$P \circ Q \geq P \min_{i \in [n]} Q_{ii} \tag{15}$$

Applying this to each term, then using Weyl's inequality for the sum :

$$JJ^T \geq \sum_{k=0}^{L-1} F_k F_k^T \circ B_{k+1} B_{k+1}^T \tag{16}$$

$$\geq \sum_{k=0}^{L-1} F_k F_k^T \min_{i \in [N]} \|(B_{k+1})_{i:}\|_2^2 \tag{17}$$

This reduction separates the problem into :

1. The **minimum eigenvalues of feature Gram matrices** : $F_k F_k^T$
2. The **minimum row norms of derivative matrices** : $\min_{i \in [N]} \|(B_{k+1})_{i:}\|_2^2$

11.2.3 • Step 3 : Bounding Feature Matrix Eigenvalues

To bound $F_k F_k^T$, the strategy uses Chernoff-type matrix concentration. The fundamental bound is given by : [Matrix Concentration for Features] Let $\lambda = \mathbb{E}_{w\sim\mathcal{N}(0,\beta_k^2\mathbf{I}_{n_{k-1}})}[\sigma(F_{k-1}w)\sigma(F_{k-1}w)^T]$. If

$$n_k \geq \max\left(N, cQ\max(1,\log(4Q))\log\frac{N}{\delta}\right) \tag{18}$$

where $Q = \frac{\beta_k^2\|F_{k-1}\|_F^2}{\lambda}$, then with probability at least $1-\delta$:

$$F_k{}^2 \geq \frac{n_k\lambda}{4} \tag{19}$$

This lemma assumes that :

— The width n_k of layer k is sufficiently large compared to the number of samples N

— The feature matrix F_{k-1} from the previous layer has bounded Frobenius norm

These conditions ensure concentration of the minimum singular value of the feature matrix around its expected value.

11.2.4 • Step 4 : Spectral Analysis via Hermite Expansion

To bound λ, we use the Hermite expansion of the ReLU. and exploiting the homogeneity of σ (unless, generalized hermite) :

$$\lambda = \mathbb{E}[\sigma(F_{k-1}w)\sigma(F_{k-1}w)^T] \tag{20}$$

$$= D\mathbb{E}[\sigma(\hat{F}_{k-1}w)\sigma(\hat{F}_{k-1}w)^T]D \tag{21}$$

where $D = \mathrm{diag}(\|\{F_{k-1}\}_{i:}\|_2)$ and $\hat{F}_{k-1} = D^{-1}F_{k-1}$.

Applying the Hermite expansion $\sigma(x) = \sum_{r=0}^{\infty}\mu_r(\sigma)H_r(x)$:

$$\mathbb{E}[\sigma(\hat{F}_{k-1}w)\sigma(\hat{F}_{k-1}w)^T] = \mu_0(\sigma)^2\mathbf{1}_N\mathbf{1}_N^T + \sum_{s=1}^{\infty}\mu_s(\sigma)^2(\hat{F}_{k-1}^{*s})(\hat{F}_{k-1}^{*s})^T \tag{22}$$

where $\hat{F}_{k-1}^{*s}$ represents the s-th Khatri-Rao power of $\hat{F}_{k-1}$.

11.2.5 • Step 5 : Reduction to Khatri-Rao Powers

For an integer $r \geq 2$, we can lower bound :

$$\lambda \geq \mu_r(\sigma)^2 D(\hat{F}_{k-1}^{*r})(\hat{F}_{k-1}^{*r})^T D = \mu_r(\sigma)^2\frac{(F_{k-1}^{*r})(F_{k-1}^{*r})^T}{\max_{i\in[N]}\|(F_{k-1})_{i:}\|_2^{2(r-1)}} \tag{23}$$

11.2.6 • Step 6 : Feature Centering and Hadamard Power Analysis

To analyze $(F_{k-1}^{*r})(F_{k-1}^{*r})^T = (F_{k-1}F_{k-1}^T)^{\circ r}$, we introduce centered features $\tilde{F}_{k-1} = F_{k-1} - \mathbb{E}_X[F_{k-1}]$.

Let $\mu = \mathbb{E}_x[f_{k-1}(x)] \in^{n_{k-1}}$ and $\Lambda = \mathrm{diag}(F_{k-1}\mu - \|\mu\|_2^2\mathbf{1}_N)$. Then :

$$(F_{k-1}^{*r})(F_{k-1}^{*r})^T = (F_{k-1}F_{k-1}^T)^{\circ r} \succeq \left(\tilde{F}_{k-1}\tilde{F}_{k-1}^T - \frac{\Lambda\mathbf{1}_N\mathbf{1}_N^T\Lambda}{\|\mu\|_2^2}\right)^{\circ r} \tag{24}$$

11.2.7 • Step 7 : Application of Gershgorin Circle Theorem

To bound the minimum eigenvalue of $(\tilde{F}_{k-1}\tilde{F}_{k-1}^T)^{\circ r}$, we apply the Gershgorin circle theorem :

$$(\tilde{F}_{k-1}^{*r})(\tilde{F}_{k-1}^{*r})^T \geq \min_{i\in[N]} \|(\tilde{F}_{k-1})_{i:}\|_2^{2r} - N \max_{i\neq j} |\langle (\tilde{F}_{k-1})_{i:}, (\tilde{F}_{k-1})_{j:}\rangle|^r \tag{25}$$

Using concentration properties of centered features, we show :

$$\|(\tilde{F}_{k-1})_{i:}\|_2^{2r} = \Theta\left(\left(d\prod_{l=1}^{k-1} n_l\beta_l^2\right)^r\right) \tag{26}$$

$$N \max_{i\neq j} |\langle (\tilde{F}_{k-1})_{i:}, (\tilde{F}_{k-1})_{j:}\rangle|^r = o\left(\left(d\prod_{l=1}^{k-1} n_l\beta_l^2\right)^r\right) \tag{27}$$

with exponentially high probability.

11.2.8 • Step 8 : Bounding Derivative Term Norms, very technical

To bound $\min_{i\in[N]} \|(B_{k+1})_{i:}\|_2^2$, we analyze each case :

For $k \in [L-2]$:

$$\|(B_{k+1})_{i:}\|_2^2 = \|\Sigma_{k+1}(x_i)\left(\prod_{l=k+2}^{L-1} W_l\Sigma_l(x_i)\right)W_L\|_2^2 \tag{28}$$

$$= \Theta\left(\beta_L^2 n_{k+1}\prod_{l=k+2}^{L-1} n_l\beta_l^2\right) \tag{29}$$

This bound uses some very technical and important tools :

- **Operator norm for concentration of Gaussian matrices**
- **Inductive analysis of random matrix products for every layer**
- **Properties of derivative matrices $\Sigma_l(x)$ for every layer**

11.2.9 • Step 9 : Final Assembly and Width Optimization

Combining all bounds via union bound yields :

$$\bar{K}^{(L)} \geq \sum_{k=0}^{L-1} \mu_r(\sigma)^2\Theta\left(d\prod_{l=1}^{k} n_l\beta_l^2\right)\Theta\left(\beta_L^2\prod_{l=k+1}^{L-1} n_l\beta_l^2\right) \tag{30}$$

$$= \mu_r(\sigma)^2\Theta\left(d\beta_L^2\sum_{k=0}^{L-1}\prod_{l=1}^{L-1} n_l\beta_l^2\right) \tag{31}$$

The width condition requires at least one layer to satisfy :

$$n_k = \tilde{\Omega}(N\log N), \quad \text{with } \xi_k = 1 \tag{32}$$

where ξ_k indicates layer k is "wide".

11.2.10 • Upper Bound via Diagonal Analysis

For the upper bound, we use :

$$JJ^T \leq (JJ^T)_{11} = \sum_{k=0}^{L-1} \|(F_k)_{1:}\|_2^2 \|(B_{k+1})_{1:}\|_2^2 \tag{33}$$

$$= \sum_{k=0}^{L-1} \|f_k(x_1)\|_2^2 \|(B_{k+1})_{1:}\|_2^2 \tag{34}$$

$$= \mathcal{O}\left(d\beta_L^2 \sum_{k=0}^{L-1} \prod_{l=1}^{L-1} n_l \beta_l^2\right) \tag{35}$$

11.2.11 • Final Result

Assuming $\beta_l = 1$ for simplicity and that some layer k^* satisfies $n_{k^*} = \tilde{\Omega}(N)$, the main theorem gives :

$$\mu_r(\sigma)^2 \Omega(d) \leq \bar{K}^{(L)} \leq \mathcal{O}(dL) \tag{36}$$

with probability at least $1 - \text{poly}(N) \exp(-\Omega(\min_l n_l))$.

This approach significantly reduces width requirements compared to classical methods, requiring only one wide layer rather than all layers.

12

CORRECTIONS À LARGEUR FINIE ET RÉGIMES D'APPRENTISSAGE

12.1 Développement Théorique pour les FCNN

(Présentation des théorèmes et formules pour les corrections à largeur finie dans les FCNN. Analyse des lois de scaling du reste et discussion sur les termes d'erreur, en lien avec les ensembles de matrices aléatoires QUE GOE.)

13

FRAMEWORK AND NOTATIONS

In this section, we introduce the mathematical framework and notations used throughout this document, building upon the definitions established in the work on the Neural Tangent Hierarchy (NTH) [**dyer2019asymptoticswidenetworksf**

13.1 Neural Network Architecture

We consider a fully-connected feedforward neural network with H hidden layers of width m.

- The network input is a vector $x \in \mathbb{R}^d$.
- The output of layer ℓ is denoted by $x^{(\ell)}$. For $\ell \in \{1, \dots, H\}$, it is computed recursively as :

$$x^{(\ell)} = \frac{1}{\sqrt{m}} \sigma(W^{(\ell)} x^{(\ell-1)}) \tag{37}$$

  where $x^{(0)} = x$ is the input, $W^{(\ell)}$ is the weight matrix of layer ℓ, and σ is a non-linear activation function applied component-wise.
- The final output of the network is a linear combination of the last hidden layer's output :

$$f(x, \theta) = a^T x^{(H)} \tag{38}$$

  where $a \in \mathbb{R}^m$ is the weight vector of the output layer.
- The set of all trainable parameters of the network is denoted $\theta = (\mathrm{vec}(W^{(1)}), \dots, \mathrm{vec}(W^{(H)}), a)$.
- The derivative of the activation function for layer ℓ is represented by a diagonal matrix $\sigma'_\ell(x) = \mathrm{diag}(\sigma'(W^{(\ell)} x^{(\ell-1)}))$.

13.2 The Neural Tangent Kernel (NTK)

The network is trained via gradient descent on the quadratic loss : $L(\theta) = \frac{1}{2n} \sum_{\alpha=1}^{n} (f(x_\alpha, \theta) - y_\alpha)^2$. The dynamics of the network function $f(x, \theta_t)$ during continuous-time training (gradient flow) are governed by the Neural Tangent Kernel (NTK), denoted $K_t^{(2)}$.

$$\frac{d}{dt} f(x, \theta_t) = -\frac{1}{n} \sum_{\beta=1}^{n} K_t^{(2)}(x, x_\beta)(f(x_\beta, \theta_t) - y_\beta) \tag{39}$$

The NTK is defined as the inner product of the gradients of the output function with respect to the network parameters :

$$K_t^{(2)}(x_\alpha, x_\beta) := \langle \nabla_\theta f(x_\alpha, \theta_t), \nabla_\theta f(x_\beta, \theta_t) \rangle \tag{40}$$

It can be decomposed as a sum over the network layers :

$$K_t^{(2)}(x_\alpha, x_\beta) = \langle x_\alpha^{(H)}, x_\beta^{(H)} \rangle + \sum_{\ell=1}^{H} \langle G_\alpha^{(\ell)}, G_\beta^{(\ell)} \rangle \langle x_\alpha^{(\ell-1)}, x_\beta^{(\ell-1)} \rangle \tag{41}$$

where the vector $G_\mu^{(\ell)}$ represents the backpropagated gradient up to layer ℓ :

$$G_\mu^{(\ell)} = \frac{1}{\sqrt{m}} \sigma'_\ell(x_\mu)(W^{(\ell+1)})^T \cdots \frac{1}{\sqrt{m}} \sigma'_H(x_\mu) a_t \tag{42}$$

13.3 The Neural Tangent Hierarchy (NTH)

For finite-width networks ($m < \infty$), the NTK $K_t^{(2)}$ is not constant but evolves during training. Its dynamics are dictated by the higher-order kernel $K_t^{(3)}$. This gives rise to an infinite sequence of coupled ordinary differential equations, known as the Neural Tangent Hierarchy (NTH) :

$$\frac{d}{dt} K_t^{(r)}(x_{\alpha_1}, \dots, x_{\alpha_r}) = -\frac{1}{n} \sum_{\gamma=1}^{n} K_t^{(r+1)}(x_{\alpha_1}, \dots, x_{\alpha_r}, x_\gamma)(f_t(x_\gamma) - y_\gamma) \tag{43}$$

valid for all $r \geq 2$. The higher-order kernel $K^{(r+1)}$ is defined recursively as the derivative of $K^{(r)}$ with respect to the parameters, contracted with the gradient of the network output :

$$K_t^{(r+1)}(x_1, \ldots, x_{r+1}) := \langle \nabla_\theta K_t^{(r)}(x_1, \ldots, x_r), \nabla_\theta f_t(x_{r+1}) \rangle \tag{44}$$

This hierarchy captures the complete training dynamics beyond the infinite-width approximation where only $K^{(2)}$ is considered and is constant. The purpose of this document is to derive and analyze the explicit form of $K^{(3)}$, which represents the first correction to the NTK dynamics.

14
OTHER PROOF

In the framework of the Neural Tangent Hierarchy (NTH), the kernel $K^{(3)}$ arises as the operator governing the time evolution of the Neural Tangent Kernel $K^{(2)}$. The dynamics of $K^{(2)}$ under gradient flow are given by :

$$\frac{d}{dt} K_t^{(2)}(x_\alpha, x_\beta) = -\frac{1}{n} \sum_{\gamma=1}^{n} K_t^{(3)}(x_\alpha, x_\beta, x_\gamma)(f_t(x_\gamma) - y_\gamma) \tag{45}$$

where $K_t^{(3)}(x_\alpha, x_\beta, x_\gamma)$ is defined as the contraction of the gradient of $K_t^{(2)}(x_\alpha, x_\beta)$ with respect to the network parameters θ with the gradient of the network output $f_t(x_\gamma)$:

$$K_t^{(3)}(x_\alpha, x_\beta, x_\gamma) := \langle \nabla_\theta K_t^{(2)}(x_\alpha, x_\beta), \nabla_\theta f_t(x_\gamma) \rangle. \tag{46}$$

Let's derive the explicit formula for $K_t^{(3)}$. We start from the expression for $K_t^{(2)}(x_\alpha, x_\beta)$:

$$K_t^{(2)}(x_\alpha, x_\beta) = \langle x_\alpha^{(H)}, x_\beta^{(H)} \rangle + \sum_{\ell=1}^{H} \langle G_\alpha^{(\ell)}, G_\beta^{(\ell)} \rangle \langle x_\alpha^{(\ell-1)}, x_\beta^{(\ell-1)} \rangle \tag{47}$$

where we use the notation :

- $x_\mu^{(p)} = \frac{1}{\sqrt{m}} \sigma(W^{(p)} x_\mu^{(p-1)})$ for $p \geq 1$, and $x_\mu^{(0)} = x_\mu$.
- $G_\mu^{(\ell)} = \frac{1}{\sqrt{m}} \sigma'_\ell(x_\mu)(W^{(\ell+1)})^T \cdots \frac{1}{\sqrt{m}} \sigma'_H(x_\mu) a_t$, with $\sigma'_\ell(x_\mu) = \mathrm{diag}(\sigma'(W^{(\ell)} x_\mu^{(\ell-1)}))$.

Let $\delta_\gamma(\cdot) := \langle \nabla_\theta(\cdot), \nabla_\theta f_t(x_\gamma) \rangle$ denote the directional derivative along $\nabla_\theta f_t(x_\gamma)$. Then $K^{(3)}_{\alpha\beta\gamma} = \delta_\gamma(K^{(2)}_{\alpha\beta})$. Applying the product rule, we get :

$$K^{(3)}_{\alpha\beta\gamma} = \delta_\gamma \langle x_\alpha^{(H)}, x_\beta^{(H)} \rangle + \sum_{\ell=1}^{H} \delta_\gamma \left(\langle G_\alpha^{(\ell)}, G_\beta^{(\ell)} \rangle \langle x_\alpha^{(\ell-1)}, x_\beta^{(\ell-1)} \rangle \right) \tag{48}$$

Each term expands further, following $\delta_\gamma \langle A, B \rangle = \langle \delta_\gamma A, B \rangle + \langle A, \delta_\gamma B \rangle$. This results in a sum over all components of $K^{(2)}_{\alpha\beta}$, where each component is acted upon by δ_γ. The core of the derivation lies in computing δ_γ for the building blocks $x_\mu^{(p)}$ and $G_\mu^{(p)}$.

The recursive formula for the derivative of the activations is :

$$\delta_\gamma x_\mu^{(p)} = \frac{1}{\sqrt{m}} \sigma'_p(x_\mu) \left(W^{(p)}(\delta_\gamma x_\mu^{(p-1)}) + \langle x_\gamma^{(p-1)}, x_\mu^{(p-1)} \rangle G_\gamma^{(p)} \right) \tag{49}$$

with the base case $\delta_\gamma x_\mu^{(0)} = \delta_\gamma x_\mu = 0$.

The derivative of the backpropagated vectors $G_\mu^{(\ell)}$ is given by a sum of terms, where δ_γ acts on one factor at a time :

$$\delta_\gamma G_\mu^{(\ell)} = \sum_{p=\ell}^{H} \left(\frac{1}{\sqrt{m}} \sigma'_\ell(x_\mu) \cdots \delta_\gamma(\frac{1}{\sqrt{m}}(W^{(p+1)})^T) \cdots \sigma'_H(x_\mu) a_t \right) \tag{50}$$

$$+ \sum_{p=\ell}^{H} \left(\frac{1}{\sqrt{m}} \sigma'_\ell(x_\mu) \cdots \delta_\gamma(\sigma'_p(x_\mu)) \cdots \sigma'_H(x_\mu) a_t \right) \tag{51}$$

$$+ \left(\frac{1}{\sqrt{m}} \sigma'_\ell(x_\mu) \cdots \sigma'_H(x_\mu)(\delta_\gamma a_t) \right) \tag{52}$$

where

- $\delta_\gamma a_t = x_\gamma^{(H)}$
- The term $\delta_\gamma((W^{(p+1)})^T)$ corresponds to a rank-1 update which, when contracted in context, yields terms involving $G_\gamma^{(p+1)}$ and inner products of activations.
- $\delta_\gamma(\sigma'_p(x_\mu)) = \sigma''_p(x_\mu)\mathrm{diag}(\delta_\gamma(W^{(p)} x_\mu^{(p-1)}/\sqrt{m}))$. This term vanishes for ReLU activation functions as $\sigma'' = 0$ almost everywhere.

Combining these rules gives the complete formula for $K_t^{(3)}(x_\alpha, x_\beta, x_\gamma)$. It is a sum of many terms, each corresponding to differentiating one part of $K_t^{(2)}(x_\alpha, x_\beta)$. For clarity, we write it as a sum of contributions :

$$K^{(3)}_{\alpha\beta\gamma} = K^{(3,\mathrm{out})}_{\alpha\beta\gamma} + \sum_{\ell=1}^{H} \left(K^{(3,G)}_{\alpha\beta\gamma,\ell} + K^{(3,x)}_{\alpha\beta\gamma,\ell} \right) \tag{53}$$

where

$$K^{(3,\mathrm{out})}_{\alpha\beta\gamma} = \langle \delta_\gamma x_\alpha^{(H)}, x_\beta^{(H)} \rangle + \langle x_\alpha^{(H)}, \delta_\gamma x_\beta^{(H)} \rangle \tag{54}$$

$$K^{(3,G)}_{\alpha\beta\gamma,\ell} = \left(\langle \delta_\gamma G_\alpha^{(\ell)}, G_\beta^{(\ell)} \rangle + \langle G_\alpha^{(\ell)}, \delta_\gamma G_\beta^{(\ell)} \rangle \right) \langle x_\alpha^{(\ell-1)}, x_\beta^{(\ell-1)} \rangle \tag{55}$$

$$K^{(3,x)}_{\alpha\beta\gamma,\ell} = \langle G_\alpha^{(\ell)}, G_\beta^{(\ell)} \rangle \left(\langle \delta_\gamma x_\alpha^{(\ell-1)}, x_\beta^{(\ell-1)} \rangle + \langle x_\alpha^{(\ell-1)}, \delta_\gamma x_\beta^{(\ell-1)} \rangle \right) \tag{56}$$

The terms $\delta_\gamma x_\mu^{(p)}$ and $\delta_\gamma G_\mu^{(p)}$ are expanded using the recursive rules above, yielding a full, albeit lengthy, expression. This hierarchical structure is characteristic of the NTH framework.

15
EXTENDED FORMULA FOR $K^{(3)}$

To obtain a complete expression for $K^{(3)}_{\alpha\beta\gamma}$, we expand the terms $\delta_\gamma x_\mu^{(p)}$ and $\delta_\gamma G_\mu^{(\ell)}$ using their recursive definitions. We assume a ReLU activation function, so $\sigma'' = 0$ almost everywhere.

15.1 Development of Recursive Terms

The term $\delta_\gamma x_\mu^{(p)}$, which represents the variation of activation $x_\mu^{(p)}$ along the gradient of output $f_t(x_\gamma)$, can be expanded as a sum over previous layers :

$$\delta_\gamma x_\mu^{(p)} = \sum_{j=1}^{p} \left(\prod_{k=j+1}^{p} \frac{\sigma'_k(x_\mu) W^{(k)}}{\sqrt{m}} \right) \frac{\sigma'_j(x_\mu)}{\sqrt{m}} \langle x_\gamma^{(j-1)}, x_\mu^{(j-1)} \rangle G_\gamma^{(j)} \tag{57}$$

where the product $\prod_{k=j+1}^{p}$ is an ordered product of Jacobian matrices.

Similarly, the term $\delta_\gamma G_\mu^{(\ell)}$, the variation of the backpropagated gradient vector, decomposes into a sum over subsequent layers :

$$\delta_\gamma G_\mu^{(\ell)} = \sum_{p=\ell}^{H-1} \left(\prod_{k=\ell}^{p} \frac{(W^{(k+1)})^T \sigma'_{k+1}(x_\mu)}{\sqrt{m}} \right) \frac{1}{m} G_\gamma^{(p+1)} \langle x_\gamma^{(p)}, x_\mu^{(p)} \rangle \tag{58}$$

$$+ \left(\prod_{k=\ell}^{H-1} \frac{(W^{(k+1)})^T \sigma'_{k+1}(x_\mu)}{\sqrt{m}} \right) \frac{x_\gamma^{(H)}}{\sqrt{m}} \tag{59}$$

The first term captures the effect of varying weights $W^{(\ell+1)}, \dots, W^{(H)}$, and the second term that of varying the output vector a_t.

15.2 Complete Expression

Substituting these expressions into equations (18), (19) and (20) yields the complete formula for $K^{(3)}_{\alpha\beta\gamma}$. It is a sum of $2H^2 + 2H$ terms.

$$\begin{aligned} K^{(3)}_{\alpha\beta\gamma} =& \sum_{j=1}^{H} \left(\langle \mathcal{J}_\alpha^{(H\to j)} \mathcal{T}_{\gamma\alpha}^{(j)}, x_\beta^{(H)} \rangle + \langle x_\alpha^{(H)}, \mathcal{J}_\beta^{(H\to j)} \mathcal{T}_{\gamma\beta}^{(j)} \rangle \right) \\ &+ \sum_{\ell=1}^{H} \langle x_\alpha^{(\ell-1)}, x_\beta^{(\ell-1)} \rangle \sum_{p=\ell}^{H-1} \left(\langle \mathcal{B}_\alpha^{(\ell\to p)} \mathcal{U}_{\gamma\alpha}^{(p)}, G_\beta^{(\ell)} \rangle + \langle G_\alpha^{(\ell)}, \mathcal{B}_\beta^{(\ell\to p)} \mathcal{U}_{\gamma\beta}^{(p)} \rangle \right) \\ &+ \sum_{\ell=1}^{H} \langle x_\alpha^{(\ell-1)}, x_\beta^{(\ell-1)} \rangle \left(\langle \mathcal{B}_\alpha^{(\ell\to H-1)} \frac{x_\gamma^{(H)}}{\sqrt{m}}, G_\beta^{(\ell)} \rangle + \langle G_\alpha^{(\ell)}, \mathcal{B}_\beta^{(\ell\to H-1)} \frac{x_\gamma^{(H)}}{\sqrt{m}} \rangle \right) \\ &+ \sum_{\ell=1}^{H} \langle G_\alpha^{(\ell)}, G_\beta^{(\ell)} \rangle \sum_{j=1}^{\ell-1} \left(\langle \mathcal{J}_\alpha^{(\ell-1\to j)} \mathcal{T}_{\gamma\alpha}^{(j)}, x_\beta^{(\ell-1)} \rangle + \langle x_\alpha^{(\ell-1)}, \mathcal{J}_\beta^{(\ell-1\to j)} \mathcal{T}_{\gamma\beta}^{(j)} \rangle \right) \end{aligned} \tag{60}$$

where we have defined for readability :

- Forward propagators (Jacobian) : $\mathcal{J}_\mu^{(p\to j)} = \prod_{k=j+1}^{p} \frac{\sigma'_k(x_\mu) W^{(k)}}{\sqrt{m}}$
- Forward source terms : $\mathcal{T}_{\gamma\mu}^{(j)} = \frac{\sigma'_j(x_\mu)}{\sqrt{m}} \langle x_\gamma^{(j-1)}, x_\mu^{(j-1)} \rangle G_\gamma^{(j)}$
- Backward propagators : $\mathcal{B}_\mu^{(\ell\to p)} = \prod_{k=\ell}^{p} \frac{(W^{(k+1)})^T \sigma'_{k+1}(x_\mu)}{\sqrt{m}}$
- Backward source terms : $\mathcal{U}_{\gamma\mu}^{(p)} = \frac{1}{m} G_\gamma^{(p+1)} \langle x_\gamma^{(p)}, x_\mu^{(p)} \rangle$

This formula, while complex, represents the complete hierarchical structure of the first-order correction to the NTK dynamics.

16

FULLY EXPANDED NON-RECURSIVE FORMULA FOR $K^{(3)}$

Here, we present the complete, non-recursive expression for $K^{(3)}_{\alpha\beta\gamma}$. This formula is obtained by substituting the expanded expressions for the directional derivatives $\delta_\gamma x^{(p)}_\mu$ and $\delta_\gamma G^{(\ell)}_\mu$ into the decomposed formula for $K^{(3)}$. This makes the dependency on the network parameters (weights $W^{(k)}$ and output vectors a_t) and activations at each layer fully explicit. We continue to assume a ReLU activation function, which sets second derivatives of σ to zero almost everywhere.

The final expression is a sum of four main components, corresponding to the differentiation of the output layer activations, the backward vectors, and the hidden layer activations :

$$
\begin{aligned}
K^{(3)}_{\alpha\beta\gamma} = \\
&\sum_{j=1}^{H}\Bigg(\Bigg\langle\left(\prod_{k=j+1}^{H}\frac{\sigma'_k(x_\alpha)W^{(k)}}{\sqrt{m}}\right)\frac{\sigma'_j(x_\alpha)}{\sqrt{m}}\langle x^{(j-1)}_\gamma, x^{(j-1)}_\alpha\rangle G^{(j)}_\gamma, x^{(H)}_\beta\Bigg\rangle \\
&\quad+\Bigg\langle x^{(H)}_\alpha, \left(\prod_{k=j+1}^{H}\frac{\sigma'_k(x_\beta)W^{(k)}}{\sqrt{m}}\right)\frac{\sigma'_j(x_\beta)}{\sqrt{m}}\langle x^{(j-1)}_\gamma, x^{(j-1)}_\beta\rangle G^{(j)}_\gamma\Bigg\rangle\Bigg) \\
&+\sum_{\ell=1}^{H}\langle x^{(\ell-1)}_\alpha, x^{(\ell-1)}_\beta\rangle\Bigg[\\
&\sum_{p=\ell}^{H-1}\Bigg(\Bigg\langle\left(\prod_{k=\ell}^{p}\frac{(W^{(k+1)})^T\sigma'_{k+1}(x_\alpha)}{\sqrt{m}}\right)\frac{G^{(p+1)}_\gamma\langle x^{(p)}_\gamma, x^{(p)}_\alpha\rangle}{m}, G^{(\ell)}_\beta\Bigg\rangle \\
&\quad+\Bigg\langle G^{(\ell)}_\alpha, \left(\prod_{k=\ell}^{p}\frac{(W^{(k+1)})^T\sigma'_{k+1}(x_\beta)}{\sqrt{m}}\right)\frac{G^{(p+1)}_\gamma\langle x^{(p)}_\gamma, x^{(p)}_\beta\rangle}{m}\Bigg\rangle\Bigg) \\
&+\Bigg\langle\left(\prod_{k=\ell}^{H-1}\frac{(W^{(k+1)})^T\sigma'_{k+1}(x_\alpha)}{\sqrt{m}}\right)\frac{x^{(H)}_\gamma}{\sqrt{m}}, G^{(\ell)}_\beta\Bigg\rangle \\
&+\Bigg\langle G^{(\ell)}_\alpha, \left(\prod_{k=\ell}^{H-1}\frac{(W^{(k+1)})^T\sigma'_{k+1}(x_\beta)}{\sqrt{m}}\right)\frac{x^{(H)}_\gamma}{\sqrt{m}}\Bigg\rangle\Bigg] \\
&+\sum_{\ell=1}^{H}\langle G^{(\ell)}_\alpha, G^{(\ell)}_\beta\rangle\sum_{j=1}^{\ell-1}\Bigg(\Bigg\langle\left(\prod_{k=j+1}^{\ell-1}\frac{\sigma'_k(x_\alpha)W^{(k)}}{\sqrt{m}}\right)\frac{\sigma'_j(x_\alpha)}{\sqrt{m}}\langle x^{(j-1)}_\gamma, x^{(j-1)}_\alpha\rangle G^{(j)}_\gamma, x^{(\ell-1)}_\beta\Bigg\rangle \\
&\quad+\Bigg\langle x^{(\ell-1)}_\alpha, \left(\prod_{k=j+1}^{\ell-1}\frac{\sigma'_k(x_\beta)W^{(k)}}{\sqrt{m}}\right)\frac{\sigma'_j(x_\beta)}{\sqrt{m}}\langle x^{(j-1)}_\gamma, x^{(j-1)}_\beta\rangle G^{(j)}_\gamma\Bigg\rangle\Bigg)
\end{aligned}
\tag{61}
$$

In this expression, the products $\prod$ represent ordered matrix products. The empty product $\prod_{k=p+1}^{p}$ is defined as the identity matrix. This detailed formula lays bare the intricate dependencies of the NTK dynamics on the network's architecture and parameters at finite width.

17
RECURSIVE FORMULA FOR $K^{(3)}$ AS A FUNCTION OF DEPTH H

To analyze the influence of network depth, it is useful to establish a recursive relation for $K^{(3)}$. Let $K^{(3,H)}_{\alpha\beta\gamma}$ denote the kernel for a network with H layers. The goal is to express $K^{(3,H+1)}_{\alpha\beta\gamma}$ in terms of quantities computed for a network with H layers.

The starting point is the decomposition of $K^{(2,H+1)}_{\alpha\beta}$, the NTK for a network with $H+1$ layers :

$$K^{(2,H+1)}_{\alpha\beta} = \underbrace{\left(\langle x^{(H)}_\alpha, x^{(H)}_\beta\rangle + \sum_{\ell=1}^{H}\langle G'^{(\ell)}_\alpha, G'^{(\ell)}_\beta\rangle\langle x^{(\ell-1)}_\alpha, x^{(\ell-1)}_\beta\rangle\right)}_{K^{(2,H)}_{\alpha\beta} \text{ with } a'_t} + T_{H+1} + C_{H+1} \tag{62}$$

where :
- $x^{(H+1)}_\mu = \frac{1}{\sqrt{m}}\sigma(W^{(H+1)}x^{(H)}_\mu)$.
- $a'_t = \frac{1}{\sqrt{m}}(W^{(H+1)})^T\sigma'_{H+1}a_t$ is an "effective output vector" that replaces a_t in the computations of $G^{(\ell)}$ for $\ell \leq H$.
- $T_{H+1} = \langle G^{(H+1)}_\alpha, G^{(H+1)}_\beta\rangle\langle x^{(H)}_\alpha, x^{(H)}_\beta\rangle$ is the term from the new layer $H+1$.
- $C_{H+1} = \langle x^{(H+1)}_\alpha, x^{(H+1)}_\beta\rangle - \langle a'_t, a'_t\rangle\langle x^{(H)}_\alpha, x^{(H)}_\beta\rangle$ is a correction term.

The vectors $G'^{(\ell)}$ are the backpropagated gradients in a network with H layers but with a'_t as output weights, while $G^{(H+1)}$ is the gradient for layer $H+1$.

Applying the directional derivative operator $\delta_\gamma(\cdot) = \langle\nabla_\theta(\cdot), \nabla_\theta f_t(x_\gamma)\rangle$, we obtain the recursion for $K^{(3)}$:

$$K^{(3,H+1)}_{\alpha\beta\gamma} = \delta_\gamma\left(K^{(2,H)}_{\alpha\beta,a'_t}\right) + \delta_\gamma(T_{H+1}) + \delta_\gamma(C_{H+1}) \tag{63}$$

The term $\delta_\gamma\left(K^{(2,H)}_{\alpha\beta,a'_t}\right)$ is analogous to $K^{(3,H)}_{\alpha\beta\gamma}$, but with additional dependencies through a'_t. The other terms expand as follows :

$$\delta_\gamma(T_{H+1}) = \langle\delta_\gamma G^{(H+1)}_\alpha, G^{(H+1)}_\beta\rangle\langle x^{(H)}_\alpha, x^{(H)}_\beta\rangle + \dots \tag{64}$$

$$\delta_\gamma(C_{H+1}) = \langle\delta_\gamma x^{(H+1)}_\alpha, x^{(H+1)}_\beta\rangle - \langle\delta_\gamma a'_t, a'_t\rangle\langle x^{(H)}_\alpha, x^{(H)}_\beta\rangle + \dots \tag{65}$$

This decomposition allows us to separate the effect of adding a layer by isolating terms specific to the new layer (T_{H+1}, C_{H+1}) and the modification of previous layer terms through a'_t.

18
FRAMEWORK AND NOTATIONS

In this section, we introduce the mathematical framework and notations used throughout this document, building upon the definitions established in the work on the Neural Tangent Hierarchy (NTH) [**dyer2019asymptoticswidenetworksf**

18.1 Neural Network Architecture

We consider a fully-connected feedforward neural network with H hidden layers of width m.

- The network input is a vector $x \in \mathbb{R}^d$.
- The output of layer ℓ is denoted by $x^{(\ell)}$. For $\ell \in \{1, \dots, H\}$, it is computed recursively as :

$$x^{(\ell)} = \frac{1}{\sqrt{m}} \sigma(W^{(\ell)} x^{(\ell-1)}) \tag{66}$$

  where $x^{(0)} = x$ is the input, $W^{(\ell)}$ is the weight matrix of layer ℓ, and σ is a non-linear activation function applied component-wise.
- The final output of the network is a linear combination of the last hidden layer's output :

$$f(x, \theta) = a^T x^{(H)} \tag{67}$$

  where $a \in \mathbb{R}^m$ is the weight vector of the output layer.
- The set of all trainable parameters of the network is denoted $\theta = (\mathrm{vec}(W^{(1)}), \dots, \mathrm{vec}(W^{(H)}), a)$.
- The derivative of the activation function for layer ℓ is represented by a diagonal matrix $\sigma'_\ell(x) = \mathrm{diag}(\sigma'(W^{(\ell)} x^{(\ell-1)}))$.

18.2 The Neural Tangent Kernel (NTK)

The network is trained via gradient descent on the quadratic loss : $L(\theta) = \frac{1}{2n} \sum_{\alpha=1}^{n} (f(x_\alpha, \theta) - y_\alpha)^2$. The dynamics of the network function $f(x, \theta_t)$ during continuous-time training (gradient flow) are governed by the Neural Tangent Kernel (NTK), denoted $K_t^{(2)}$.

$$\frac{d}{dt} f(x, \theta_t) = -\frac{1}{n} \sum_{\beta=1}^{n} K_t^{(2)}(x, x_\beta)(f(x_\beta, \theta_t) - y_\beta) \tag{68}$$

The NTK is defined as the inner product of the gradients of the output function with respect to the network parameters :

$$K_t^{(2)}(x_\alpha, x_\beta) := \langle \nabla_\theta f(x_\alpha, \theta_t), \nabla_\theta f(x_\beta, \theta_t) \rangle \tag{69}$$

It can be decomposed as a sum over the network layers :

$$K_t^{(2)}(x_\alpha, x_\beta) = \langle x_\alpha^{(H)}, x_\beta^{(H)} \rangle + \sum_{\ell=1}^{H} \langle G_\alpha^{(\ell)}, G_\beta^{(\ell)} \rangle \langle x_\alpha^{(\ell-1)}, x_\beta^{(\ell-1)} \rangle \tag{70}$$

where the vector $G_\mu^{(\ell)}$ represents the backpropagated gradient up to layer ℓ :

$$G_\mu^{(\ell)} = \frac{1}{\sqrt{m}} \sigma'_\ell(x_\mu)(W^{(\ell+1)})^T \cdots \frac{1}{\sqrt{m}} \sigma'_H(x_\mu) a_t \tag{71}$$

18.3 The Neural Tangent Hierarchy (NTH)

For finite-width networks ($m < \infty$), the NTK $K_t^{(2)}$ is not constant but evolves during training. Its dynamics are dictated by the higher-order kernel $K_t^{(3)}$. This gives rise to an infinite sequence of coupled ordinary differential equations, known as the Neural Tangent Hierarchy (NTH) :

$$\frac{d}{dt} K_t^{(r)}(x_{\alpha_1}, \dots, x_{\alpha_r}) = -\frac{1}{n} \sum_{\gamma=1}^{n} K_t^{(r+1)}(x_{\alpha_1}, \dots, x_{\alpha_r}, x_\gamma)(f_t(x_\gamma) - y_\gamma) \tag{72}$$

valid for all $r \geq 2$. The higher-order kernel $K^{(r+1)}$ is defined recursively as the derivative of $K^{(r)}$ with respect to the parameters, contracted with the gradient of the network output :

$$K_t^{(r+1)}(x_1, \ldots, x_{r+1}) := \langle \nabla_\theta K_t^{(r)}(x_1, \ldots, x_r), \nabla_\theta f_t(x_{r+1}) \rangle \tag{73}$$

This hierarchy captures the complete training dynamics beyond the infinite-width approximation where only $K^{(2)}$ is considered and is constant. The purpose of this document is to derive and analyze the explicit form of $K^{(3)}$, which represents the first correction to the NTK dynamics.

19
INTRODUCTION

This document analyzes the behavior of the entries of the $K^{(3)}$ tensor as a function of the network depth, denoted by L (or H in the code), for a fixed network width m. The analysis is based on the formulas from the Neural Tangent Hierarchy (NTH) framework. We aim to understand if the tensor entries decay as the depth increases, and specifically, if this decay is exponential.

20
SCALING OF CORE COMPONENTS IN THE NTK FORMALISM

We consider a Multi-Layer Perceptron (MLP) of depth L and uniform width m. The weights $W^{(\ell)}$ for $\ell = 1, \ldots, L$ are $m \times m$ matrices, and the output weights a are in $\mathbb{R}^m$. We use the NTK scaling, where weights are initialized i.i.d. from a distribution with variance c_W^2/m (e.g., $\mathcal{N}(0, c_W^2/m)$). The output layer is scaled by $1/\sqrt{m}$.

The forward activations $x_\mu^{(p)}$ and backward propagated vectors $G_\mu^{(\ell)}$ are the building blocks of the NTK. The activations are defined as $x_\mu^{(p)} = \frac{1}{\sqrt{m}} \sigma(W^{(p)} x_\mu^{(p-1)})$. With proper initialization ($c_W^2 = 2$ for ReLU), the norm of the activations is stable across layers :

$$\mathbb{E}[\|x_\mu^{(p)}\|^2] \approx \|x_\mu^{(p-1)}\|^2 \tag{74}$$

Thus, we assume $\|x_\mu^{(p)}\| \sim O(1)$ for all layers p.

The backward vectors $G_\mu^{(\ell)}$ are defined recursively :

$$G_\mu^{(L)} = \frac{a}{\sqrt{m}} \tag{75}$$

$$G_\mu^{(\ell)} = \frac{(W^{(\ell+1)})^T}{\sqrt{m}} \sigma'_{\ell+1}(x_\mu) G_\mu^{(\ell+1)} \tag{76}$$

where $\sigma'_{\ell+1}$ is the diagonal matrix of activation derivatives. A crucial point is to analyze the norm of the propagation operator $\mathcal{P}_\ell = \frac{(W^{(\ell+1)})^T}{\sqrt{m}} \sigma'_{\ell+1}(x_\mu)$. Based on random matrix theory, for a large matrix W with i.i.d. entries of variance σ_w^2, its operator norm $\|W\|_{\text{op}}$ converges to $2\sigma_w\sqrt{m}$. Thus, the norm of our scaled operator is :

$$\|\mathcal{P}_\ell\|_{\text{op}} \leq \left\| \frac{W^{(\ell+1)}}{\sqrt{m}} \right\|_{\text{op}} \xrightarrow{m \to \infty} 2\sigma_w \tag{77}$$

For standard initialization schemes (e.g., He initialization with $\sigma_w^2 = 2/m_{in}$), this value is $O(1)$ but not necessarily less than 1. A naive application of the sub-multiplicative property might suggest an exponential explosion of norms, as $\|G^{(\ell)}\| \leq (2\sigma_w)^{L-\ell}\|G^{(L)}\|$.

However, this reasoning is flawed because we are multiplying a sequence of **independent** random matrices. The vector $G_\mu^{(\ell+1)}$, after being transformed by $\mathcal{P}_\ell$, is not aligned with the direction of maximal amplification of the next operator $\mathcal{P}_{\ell-1}$. The theory of products of random matrices shows that for deep networks, the norm is preserved on average if the initialization is chosen correctly to achieve a state of "dynamical isometry". This ensures that the system avoids both exponential explosion and vanishing of the signal. Therefore, we can confidently assume that the norms are stable across layers :

$$\|G_\mu^{(\ell)}\| \sim O(1) \quad \text{for all } \ell \tag{78}$$

These vectors do not exhibit a systematic decay or growth with depth.

21
ANALYSIS OF THE $K^{(3)}$ TENSOR

The $K^{(3)}$ tensor is constructed from the directional derivatives of $x_\mu^{(p)}$ and $G_\mu^{(\ell)}$, denoted $\delta_\gamma x_\mu^{(p)}$ and $\delta_\gamma G_\mu^{(\ell)}$.

21.1 Forward Derivative Term $\delta_\gamma x_\mu^{(p)}$

The recursive formula for $\delta_\gamma x_\mu^{(p)}$ is :

$$\delta_\gamma x_\mu^{(p)} = \frac{1}{\sqrt{m}}\sigma_p'(x_\mu)\left(W^{(p)}(\delta_\gamma x_\mu^{(p-1)}) + \langle x_\gamma^{(p-1)}, x_\mu^{(p-1)}\rangle G_\gamma^{(p)}\right) \tag{79}$$

with $\delta_\gamma x_\mu^{(0)} = 0$. Analyzing the norm :

$$\|\delta_\gamma x_\mu^{(p)}\| \leq \frac{1}{\sqrt{m}}\|\sigma_p'\|\left(\|W^{(p)}\|\|\delta_\gamma x_\mu^{(p-1)}\| + |\langle \dots \rangle|\|G_\gamma^{(p)}\|\right) \tag{80}$$

Using $\|W^{(p)}\| \sim O(\sqrt{m})$ and that other norms are $O(1)$, we get :

$$\|\delta_\gamma x_\mu^{(p)}\| \lesssim \|\delta_\gamma x_\mu^{(p-1)}\| + O(1/\sqrt{m}) \tag{81}$$

Starting from $\|\delta_\gamma x_\mu^{(0)}\| = 0$, we find $\|\delta_\gamma x_\mu^{(p)}\| \sim O(p/\sqrt{m})$. This indicates a linear growth with layer index p, scaled by $1/\sqrt{m}$.

21.2 Backward Derivative Term $\delta_\gamma G_\mu^{(\ell)}$

The term $\delta_\gamma G_\mu^{(\ell)}$ has a more complex structure, involving a sum over subsequent layers p from ℓ to L. A representative term in this sum is (ignoring propagators for simplicity) :

$$\text{term}_p \sim \frac{1}{m} G_\gamma^{(p+1)} \langle x_\gamma^{(p)}, x_\mu^{(p)} \rangle \tag{82}$$

This term has a factor of $1/m$. The propagation of this term back to layer ℓ involves products of matrices of the form $(W^{(k)})^T/\sqrt{m}$, which have $O(1)$ norm. Therefore, each term in the sum for $\delta_\gamma G_\mu^{(\ell)}$ is of order $O(1/m)$. Since there are $L-\ell$ such terms, we get :

$$\|\delta_\gamma G_\mu^{(\ell)}\| \sim O\left(\frac{L-\ell}{m}\right) \tag{83}$$

This term shows a linear dependency on the number of subsequent layers and is scaled by $1/m$.

21.3 Overall Scaling of $K^{(3)}$

The full expression for $K^{(3)}_{\alpha\beta\gamma}$ is a sum of terms involving scalar products of the quantities analyzed above. For example :

$$K^{(3)}_{\alpha\beta\gamma} = \langle \delta_\gamma x_\alpha^{(L)}, x_\beta^{(L)} \rangle + \langle x_\alpha^{(L)}, \delta_\gamma x_\beta^{(L)} \rangle + \dots \tag{84}$$

The dominant term comes from the forward derivative part :

$$\langle \delta_\gamma x_\alpha^{(L)}, x_\beta^{(L)} \rangle \sim O(L/\sqrt{m}) \tag{85}$$

The other terms involving $\delta_\gamma G$ are smaller, of order $O(L/m)$. Thus, for large m :

$$K^{(3)}_{\alpha\beta\gamma} \sim O(L/\sqrt{m}) \tag{86}$$

22
CONCLUSION ON DEPTH DEPENDENCE

Our analysis shows that the magnitude of the entries of the $K^{(3)}$ tensor does **not** decay with depth L. Instead, it appears to grow linearly with L. The scaling factor is $1/\sqrt{m}$.

This contradicts the initial suspicion of an exponential decay. The key observations are :

- The fundamental vectors $x^{(p)}$ and $G^{(\ell)}$ have norms that are stable across layers and do not decay.
- The derivative terms $\delta_\gamma x^{(p)}$ and $\delta_\gamma G^{(\ell)}$ introduce factors of $1/\sqrt{m}$ and $1/m$ respectively, but their recursive/summative nature leads to a linear growth with the number of layers involved (p or $L-\ell$).

Therefore, for a fixed width m, we expect the values of $K^{(3)}$ to increase with the depth L. There is no evidence of an exponential decay. The factor $1/\sqrt{m}$ controls the overall magnitude but does not introduce a decay with depth.

23

DETAILED SCALING ANALYSIS WITH RESPECT TO DEPTH H

We now provide a more detailed analysis of the scaling of $K^{(3)}_{\alpha\beta\gamma}$ with respect to the network depth H (denoted L in some contexts) and width m. This analysis is based on the fully expanded formula and relies on standard assumptions in the study of wide neural networks.

23.1 Core Assumptions and Scaling of Components

Our analysis rests on the following scaling assumptions, which are direct consequences of the NTK parameterization and standard random matrix theory results :

- **Activations and G-vectors :** The norms of forward activations and backward-propagated vectors are stable across layers : $\|x^{(p)}_\mu\| \sim O(1)$ and $\|G^{(\ell)}_\mu\| \sim O(1)$.
- **Propagators :** Due to the principle of dynamical isometry targeted by modern initialization schemes, the operator norms of the forward and backward propagators are of order one, regardless of the number of matrices in the product : $\|\mathcal{J}^{(p\to j)}_\mu\|_{\text{op}} \sim O(1)$ and $\|\mathcal{B}^{(\ell\to p)}_\mu\|_{\text{op}} \sim O(1)$.
- **Source Terms :** The scaling of the source terms can be readily determined :
  - Forward source term : $\mathcal{T}^{(j)}_{\gamma\mu} = \frac{\sigma'_j(x_\mu)}{\sqrt{m}}\langle x^{(j-1)}_\gamma, x^{(j-1)}_\mu\rangle G^{(j)}_\gamma$. Its norm scales as $\|\mathcal{T}^{(j)}_{\gamma\mu}\| \sim O(1/\sqrt{m})$.
  - Backward source term : $\mathcal{U}^{(p)}_{\gamma\mu} = \frac{1}{m}G^{(p+1)}_\gamma\langle x^{(p)}_\gamma, x^{(p)}_\mu\rangle$. Its norm scales as $\|\mathcal{U}^{(p)}_{\gamma\mu}\| \sim O(1/m)$.

23.2 Term-by-Term Analysis

We analyze the four main components of the fully expanded formula for $K^{(3)}_{\alpha\beta\gamma}$. For simplicity, we analyze one of the two symmetric terms in each summation.

1. Output Term ($K^{(3,\mathbf{out})}$) This term is given by the sum $\sum_{j=1}^{H}\langle \mathcal{J}^{(H\to j)}_\alpha \mathcal{T}^{(j)}_{\gamma\alpha}, x^{(H)}_\beta\rangle$. The magnitude of each summand is :

$$|\langle \mathcal{J}^{(H\to j)}_\alpha \mathcal{T}^{(j)}_{\gamma\alpha}, x^{(H)}_\beta\rangle| \le \|\mathcal{J}^{(H\to j)}_\alpha\|_{\text{op}} \cdot \|\mathcal{T}^{(j)}_{\gamma\alpha}\| \cdot \|x^{(H)}_\beta\| \sim O(1)\cdot O(1/\sqrt{m})\cdot O(1) = O(1/\sqrt{m})$$

Since we are summing H such terms, the total contribution of this component is :

$$K^{(3,\text{out})} \sim H \cdot O(1/\sqrt{m}) = O(H/\sqrt{m})$$

2. Backward Term from Weights ($K^{(3,G,W)}$) This term is the double summation $\sum_{\ell=1}^{H}\langle x^{(\ell-1)}_\alpha, x^{(\ell-1)}_\beta\rangle \sum_{p=\ell}^{H-1}\langle \mathcal{B}^{(\ell\to p)}_\alpha \mathcal{U}^{(p)}_{\gamma\alpha}, G$
The magnitude of each summand in the inner loop is :

$$|\langle\dots\rangle\langle\dots\rangle| \le |\langle x^{(\ell-1)}_\alpha, x^{(\ell-1)}_\beta\rangle| \cdot \|\mathcal{B}^{(\ell\to p)}_\alpha\|_{\text{op}} \cdot \|\mathcal{U}^{(p)}_{\gamma\alpha}\| \cdot \|G^{(\ell)}_\beta\| \sim O(1)\cdot O(1)\cdot O(1/m)\cdot O(1) = O(1/m)$$

This double sum contains approximately $\sum_{\ell=1}^{H}(H-\ell) \approx H^2/2$ terms. Thus, the total contribution is :

$$K^{(3,G,W)} \sim H^2 \cdot O(1/m) = O(H^2/m)$$

3. Backward Term from Output Layer ($K^{(3,G,a)}$) This term corresponds to the derivative with respect to the output weights a_t : $\sum_{\ell=1}^{H}\langle x_\alpha^{(\ell-1)}, x_\beta^{(\ell-1)}\rangle\langle \mathcal{B}_\alpha^{(\ell\to H-1)}\frac{x_\gamma^{(H)}}{\sqrt{m}}, G_\beta^{(\ell)}\rangle$. The magnitude of each summand is :

$$|\langle\dots\rangle\langle\dots\rangle| \le |\langle x_\alpha^{(\ell-1)}, x_\beta^{(\ell-1)}\rangle| \cdot \|\mathcal{B}_\alpha^{(\ell\to H-1)}\|_{\text{op}} \cdot \frac{\|x_\gamma^{(H)}\|}{\sqrt{m}} \cdot \|G_\beta^{(\ell)}\| \sim O(1)\cdot O(1)\cdot O(1/\sqrt{m})\cdot O(1) = O(1/\sqrt{m})$$

Summing H such terms, the total contribution is :

$$K^{(3,G,a)} \sim H \cdot O(1/\sqrt{m}) = O(H/\sqrt{m})$$

4. Forward Term from Activations ($K^{(3,x)}$) This is the final double summation : $\sum_{\ell=1}^{H}\langle G_\alpha^{(\ell)}, G_\beta^{(\ell)}\rangle \sum_{j=1}^{\ell-1}\langle \mathcal{J}_\alpha^{(\ell-1\to j)}\mathcal{T}_{\gamma\alpha}^{(j)}, x_\beta^{(\ell}$
The magnitude of each summand in the inner loop is :

$$|\langle\dots\rangle\langle\dots\rangle| \le |\langle G_\alpha^{(\ell)}, G_\beta^{(\ell)}\rangle| \cdot \|\mathcal{J}_\alpha^{(\ell-1\to j)}\|_{\text{op}} \cdot \|\mathcal{T}_{\gamma\alpha}^{(j)}\| \cdot \|x_\beta^{(\ell-1)}\| \sim O(1)\cdot O(1)\cdot O(1/\sqrt{m})\cdot O(1) = O(1/\sqrt{m})$$

This double sum contains approximately $\sum_{\ell=1}^{H}(\ell-1) \approx H^2/2$ terms. The total contribution is therefore :

$$K^{(3,x)} \sim H^2 \cdot O(1/\sqrt{m}) = O(H^2/\sqrt{m})$$

23.3 Overall Scaling and Conclusion

Combining the four components, we have :

$$K^{(3)}_{\alpha\beta\gamma} = O(H/\sqrt{m}) + O(H^2/m) + O(H/\sqrt{m}) + O(H^2/\sqrt{m})$$

For a sufficiently large width m, the term $O(H^2/\sqrt{m})$ will dominate the term $O(H^2/m)$. Therefore, the overall scaling behavior of the third-order kernel is dictated by the $K^{(3,x)}$ term :

$$K^{(3)}_{\alpha\beta\gamma} \sim O\left(\frac{H^2}{\sqrt{m}}\right) \tag{87}$$

This detailed analysis confirms that the magnitude of the $K^{(3)}$ tensor entries does not decay with depth. Instead, it exhibits a **quadratic growth** with the depth H. This implies that the dynamics of the NTK itself become more pronounced and complex in very deep networks, a key feature of the finite-width correction captured by the Neural Tangent Hierarchy.

24
IMPLEMENTATION OF THE NTK, AND THE CODE CORRESPONDENCE

The Python script `kernel3_empirical.py` in the github repository provides a direct, empirical implementation of the analytical formulas for the third-order kernel, $K^{(3)}$, as derived within the Neural Tangent Hierarchy framework. This implementation translates the mathematical recursions and summations into executable code.

24.1 Code Structure and Correspondence to Formulas

The core logic is encapsulated within the 'Kernel3Empirical' class. Its methods directly correspond to the mathematical objects we have analyzed :

— **`_compute_G(ell, mu)`** : This helper function computes the backward-propagated vector $G_\mu^{(\ell)}$. It implements the standard recursive definition :

$$G_\mu^{(\ell)} = \frac{(W^{(\ell+1)})^T}{\sqrt{m}} \sigma'_{\ell+1}(x_\mu) G_\mu^{(\ell+1)}$$

The recursion starts from the base case $G_\mu^{(H)} = a_t/\sqrt{m}$.

— **`_compute_delta_x(p, gamma, mu)`** : This function calculates the forward derivative term $\delta_\gamma x_\mu^{(p)}$. It follows the recursive formula derived from the chain rule for forward activations :

$$\delta_\gamma x_\mu^{(p)} = \frac{1}{\sqrt{m}} \sigma'_p(x_\mu) \left(W^{(p)} (\delta_\gamma x_\mu^{(p-1)}) + \langle x_\gamma^{(p-1)}, x_\mu^{(p-1)} \rangle G_\gamma^{(p)} \right)$$

The recursion starts from the base case $\delta_\gamma x_\mu^{(0)} = 0$.

— **`_compute_delta_G(ell, gamma, mu)`** : This method implements the more complex formula for the backward derivative term $\delta_\gamma G_\mu^{(\ell)}$. It constructs the term by summing the contributions from the derivatives of all subsequent weights ($W^{(p+1)}$ for $p \geq \ell$) and the output layer weights (a_t). Each contribution is then propagated back to layer ℓ through a series of matrix-vector products, as specified by the analytical formula.

— **`kernel3(alpha, beta, gamma)`** : This is the main public method. It computes the final scalar value of $K^{(3)}_{\alpha\beta\gamma}$ by assembling all the pieces. It mirrors the high-level decomposition :

$$K^{(3)}_{\alpha\beta\gamma} = K^{(3,\text{out})}_{\alpha\beta\gamma} + \sum_{\ell=1}^{H} \left(K^{(3,G)}_{\alpha\beta\gamma,\ell} + K^{(3,x)}_{\alpha\beta\gamma,\ell} \right)$$

It calls the helper methods to compute the necessary $\delta_\gamma x$, $\delta_\gamma G$, and G vectors and then combines them through the appropriate inner products.

24.2 Future Work : Optimization with Automatic Differentiation

The current implementation is a literal translation of the mathematical formulas. While valuable for its clarity and direct correspondence to the theory, this manual approach has drawbacks : it is computationally intensive due to explicit recursion and can be complex to debug and extend.

In the future, a more efficient and robust approach will be pursued by leveraging modern automatic differentiation (AD) frameworks, particularly JAX. The vision is to move away from implementing the analytical derivatives manually and instead let the framework compute them automatically. This strategy is at the heart of libraries like Google's `neural-tangents`.

The plan is as follows :

1. Define the second-order kernel, $K^{(2)}(x_\alpha, x_\beta)$, as a pure JAX function that depends on the network parameters θ.
2. Use JAX's function transformations, such as `jax.grad`, to compute the gradient of the kernel with respect to the parameters, $\nabla_\theta K^{(2)}_{\alpha\beta}$.
3. The third-order kernel, $K^{(3)}$, is defined as the inner product $K^{(3)}_{\alpha\beta\gamma} = \langle \nabla_\theta K^{(2)}_{\alpha\beta}, \nabla_\theta f_\gamma \rangle$. Since $\nabla_\theta f_\gamma$ can also be computed automatically with `jax.grad`, the entire $K^{(3)}$ tensor can be obtained without manually writing out its complex formula.

This AD-based approach promises significant benefits :

— **Efficiency :** JAX can compile the entire computation into a highly optimized computational graph for execution on accelerators like GPUs or TPUs.
— **Simplicity and Robustness :** The code becomes much simpler, as we only need to define the base objects (f and $K^{(2)}$), not their derivatives. This drastically reduces the risk of implementation errors.
— **Extensibility :** This methodology would make it far easier to compute even higher-order kernels, such as $K^{(4)}$, which would be practically infeasible to implement manually.

25

EMPIRICAL VALIDATION : EXPERIMENTAL SETUP AND ANALYSIS

The script `kernel3_vs_depth_finite_empirical_largeexperiments.py` is designed to empirically validate the theoretical scaling laws for $K^{(3)}$ derived in the previous sections. The primary goal is to investigate how the magnitude of the $K^{(3)}$ tensor entries depends on the network depth H and width M.

25.1 Experimental Design

The experiment is structured as a systematic grid search over a range of network configurations. The main loop iterates through different values for :

- `N_VALUES` : The number of data points.
- `D_IN_VALUES` : The input feature dimension.
- `M_VALUES` : The network width.
- `L_VALUES` : The network depth (referred to as H in our formulas).

For each configuration, the script performs the following steps :

1. **Initialization** : It generates random, normalized input data using `generate_data` and initializes the network weights according to a standard normal distribution via `init_network`.
2. **Forward Pass** : It calls `compute_features_and_derivatives` to perform a full forward pass, collecting all the necessary intermediate values : the feature maps ($x^{(p)}$) and the diagonal matrices of activation derivatives (σ'_p) for each layer.
3. **Kernel Computation** : An instance of the `Kernel3Empirical` class is created with the network weights and the results from the forward pass. The script then computes the entire $N \times N \times N$ tensor for $K^{(3)}$ by calling the `kernel3(i, j, k)` method for all unique triplets of indices, leveraging the tensor's symmetry.

25.2 Data Analysis and Connection to Theory

Once the empirical `k3_tensor` is computed, the script extracts quantitative metrics to analyze its properties.

- **Infinity Norm** : The script computes `inf_norm = M * np.max(np.abs(k3_tensor))`. The purpose of this analysis is to study the kernel's asymptotic behavior. By scaling the empirical result with a factor depending on the width M, we can check if the quantity converges to a finite, depth-dependent limit as $M \to \infty$. Our theoretical analysis predicted that the entries of $K^{(3)}$ scale as $O(H^2/\sqrt{M})$. Therefore, multiplying the maximum absolute value by $\sqrt{M}$ should result in a quantity that grows quadratically with depth H. The script's use of a scaling factor of M may be intended to test a different theoretical hypothesis or scaling regime.
- **Eigenvalue Analysis** : The script also performs a spectral analysis by examining the eigenvalues of 2D slices of the tensor (`k3_tensor[:, :, k]`). This provides deeper insight into the operator's properties than its maximum entry alone. The mean and standard deviation of these eigenvalues are computed across all slices to produce a statistical summary of the kernel's spectrum. These spectral properties are also scaled by the width M.

The data generated by this script (the computed norms and eigenvalue statistics for varying H and M) is saved and can be plotted to visually confirm or challenge the $O(H^2/\sqrt{M})$ scaling law derived theoretically. This

provides a crucial bridge between the analytical formulas of the NTH and their concrete implications for the behavior of finite-width neural networks.

26
DEEP DIVE INTO QUANTITATIVE METRICS

To bridge the gap between our theoretical analysis and the empirical results from the script, it's essential to understand precisely what the computed metrics represent and how they connect to the underlying theory.

26.1 Analysis of the Scaled Infinity Norm

The script computes `inf_norm = M * np.max(np.abs(k3_tensor))`. Let's break this down.

— **What is the Infinity Norm ?** The infinity norm of the $K^{(3)}$ tensor, denoted $\|K^{(3)}\|_\infty$, is simply the largest absolute value among all its entries :

$$\|K^{(3)}\|_\infty = \max_{\alpha,\beta,\gamma} |K^{(3)}_{\alpha\beta\gamma}|$$

This metric provides a simple way to quantify the overall magnitude of the kernel.

— **The Purpose of Scaling by Width** M : In the infinite-width limit ($M \to \infty$), the standard NTK theory shows that $K^{(3)}$ vanishes, which is why the kernel dynamics are ignored. Our analysis for finite-width networks aims to characterize precisely *how fast* it vanishes. The scaling factor is introduced to cancel out the width-dependent decay, thereby isolating the depth-dependent behavior.

— **Testing Theoretical Predictions** : Our detailed scaling analysis predicted that the dominant term in $K^{(3)}$ scales as $O(H^2/\sqrt{M})$. To empirically test this hypothesis, one should analyze the quantity $\sqrt{M}\cdot\|K^{(3)}\|_\infty$. If the theory is correct, this scaled value should converge to a limit that grows quadratically with depth (H^2) as M becomes large.

The experiment, by computing $M \cdot \|K^{(3)}\|_\infty$, is implicitly testing an alternative hypothesis where the kernel entries scale as $O(1/M)$. If our $O(1/\sqrt{M})$ theory is correct, the quantity computed in the script should not converge but instead grow like $\sqrt{M}$. By observing which of these scaled quantities (or another) stabilizes as M increases, the experiment can empirically determine the true scaling law, thus validating or falsifying different theoretical claims.

26.2 Spectral Analysis of Tensor Slices

The eigenvalue analysis provides a much deeper view into the structure of $K^{(3)}$ than the infinity norm alone. The dynamics of the NTK are given by the equation :

$$\frac{d}{dt}K^{(2)}_t = -\frac{1}{n}\sum_{\gamma=1}^{n}(f_t(x_\gamma) - y_\gamma)\cdot K^{(3)}_{t,\gamma}$$

where $K^{(3)}_{t,\gamma}$ is the $N \times N$ matrix slice of the tensor for a fixed index γ. The evolution of $K^{(2)}$ is thus a linear combination of these slice matrices.

— **Physical Meaning of Eigenvalues** : The eigenvalues of a slice matrix $K^{(3)}_\gamma$ characterize how an error at a single data point, x_γ, influences the geometry of the feature space (as captured by $K^{(2)}$). An eigenvector of this matrix represents a direction in the input space, and its corresponding eigenvalue represents the magnitude and sign of the change in that direction. A large positive eigenvalue means that

the kernel's value for pairs of points aligned with that eigenvector will increase, effectively pushing those representations further apart.

— **Mean and Standard Deviation** :
  — `mean_eigenvalues` : By averaging the eigenvalues of all slices, we obtain the "average" spectral signature of the $K^{(3)}$ operator. This tells us if there is a systematic, data-independent drift in the NTK. For instance, a consistently negative smallest eigenvalue across all slices would imply that gradient descent tends to contract the feature space along a specific direction, regardless of which data point has a large error.
  — `std_eigenvalues` : The standard deviation measures how much this spectral impact varies depending on the data point x_γ driving the update. A small standard deviation would imply that the kernel evolves in a very predictable, almost data-independent manner. Conversely, a large standard deviation suggests that the geometry of the kernel is being sculpted in a highly complex, data-dependent way, with each training example pulling the kernel in a different "direction".
— **Asymptotic Behavior** : As with the infinity norm, the eigenvalues are scaled by the width M to study their asymptotic behavior. The eigenvalues of a matrix with entries scaling as ϵ will also scale as ϵ. Therefore, if the entries of $K^{(3)}$ scale as $O(1/\sqrt{M})$, its eigenvalues should as well. The experiment provides the necessary data to check if the scaled spectrum converges to a stable, non-trivial limit as the width grows.

METTRE LES GRAPHES, les experiences, et le papier feynman diagram

27

IMPLEMENTATION OF THE NTK, AND THE CODE CORRESPONDENCE

The Python script `kernel3_empirical.py` in the github repository provides a direct, empirical implementation of the analytical formulas for the third-order kernel, $K^{(3)}$, as derived within the Neural Tangent Hierarchy framework. This implementation translates the mathematical recursions and summations into executable code.

27.1 Code Structure and Correspondence to Formulas

The core logic is encapsulated within the 'Kernel3Empirical' class. Its methods directly correspond to the mathematical objects we have analyzed :

— `_compute_G(ell, mu)` : This helper function computes the backward-propagated vector $G_\mu^{(\ell)}$. It implements the standard recursive definition :

$$G_\mu^{(\ell)} = \frac{(W^{(\ell+1)})^T}{\sqrt{m}} \sigma'_{\ell+1}(x_\mu) G_\mu^{(\ell+1)}$$

The recursion starts from the base case $G_\mu^{(H)} = a_t/\sqrt{m}$.

— `_compute_delta_x(p, gamma, mu)` : This function calculates the forward derivative term $\delta_\gamma x_\mu^{(p)}$. It follows the recursive formula derived from the chain rule for forward activations :

$$\delta_\gamma x_\mu^{(p)} = \frac{1}{\sqrt{m}} \sigma'_p(x_\mu) \left(W^{(p)} (\delta_\gamma x_\mu^{(p-1)}) + \langle x_\gamma^{(p-1)}, x_\mu^{(p-1)} \rangle G_\gamma^{(p)} \right)$$

The recursion starts from the base case $\delta_\gamma x_\mu^{(0)} = 0$.

— `_compute_delta_G(ell, gamma, mu)` : This method implements the more complex formula for the backward derivative term $\delta_\gamma G_\mu^{(\ell)}$. It constructs the term by summing the contributions from the derivatives of all subsequent weights ($W^{(p+1)}$ for $p \geq \ell$) and the output layer weights (a_t). Each contribution is then propagated back to layer ℓ through a series of matrix-vector products, as specified by the analytical formula.

— `kernel3(alpha, beta, gamma)` : This is the main public method. It computes the final scalar value of $K^{(3)}_{\alpha\beta\gamma}$ by assembling all the pieces. It mirrors the high-level decomposition :

$$K^{(3)}_{\alpha\beta\gamma} = K^{(3,\text{out})}_{\alpha\beta\gamma} + \sum_{\ell=1}^{H} \left(K^{(3,G)}_{\alpha\beta\gamma,\ell} + K^{(3,x)}_{\alpha\beta\gamma,\ell} \right)$$

It calls the helper methods to compute the necessary $\delta_\gamma x$, $\delta_\gamma G$, and G vectors and then combines them through the appropriate inner products.

27.2 Future Work : Optimization with Automatic Differentiation

The current implementation is a literal translation of the mathematical formulas. While valuable for its clarity and direct correspondence to the theory, this manual approach has drawbacks : it is computationally intensive due to explicit recursion and can be complex to debug and extend.

In the future, a more efficient and robust approach will be pursued by leveraging modern automatic differentiation (AD) frameworks, particularly JAX. The vision is to move away from implementing the analytical derivatives manually and instead let the framework compute them automatically. This strategy is at the heart of libraries like Google's `neural-tangents`.

The plan is as follows :

1. Define the second-order kernel, $K^{(2)}(x_\alpha, x_\beta)$, as a pure JAX function that depends on the network parameters θ.
2. Use JAX's function transformations, such as `jax.grad`, to compute the gradient of the kernel with respect to the parameters, $\nabla_\theta K^{(2)}_{\alpha\beta}$.
3. The third-order kernel, $K^{(3)}$, is defined as the inner product $K^{(3)}_{\alpha\beta\gamma} = \langle \nabla_\theta K^{(2)}_{\alpha\beta}, \nabla_\theta f_\gamma \rangle$. Since $\nabla_\theta f_\gamma$ can also be computed automatically with `jax.grad`, the entire $K^{(3)}$ tensor can be obtained without manually writing out its complex formula.

This AD-based approach promises significant benefits :

— **Efficiency :** JAX can compile the entire computation into a highly optimized computational graph for execution on accelerators like GPUs or TPUs.

— **Simplicity and Robustness :** The code becomes much simpler, as we only need to define the base objects (f and $K^{(2)}$), not their derivatives. This drastically reduces the risk of implementation errors.

— **Extensibility :** This methodology would make it far easier to compute even higher-order kernels, such as $K^{(4)}$, which would be practically infeasible to implement manually.

28

EMPIRICAL VALIDATION : EXPERIMENTAL SETUP AND ANALYSIS

The script `kernel3_vs_depth_finite_empirical_largeexperiments.py` is designed to empirically validate the theoretical scaling laws for $K^{(3)}$ derived in the previous sections. The primary goal is to investigate how the magnitude of the $K^{(3)}$ tensor entries depends on the network depth H and width M.

28.1 Experimental Design

The experiment is structured as a systematic grid search over a range of network configurations. The main loop iterates through different values for :

- `N_VALUES` : The number of data points.
- `D_IN_VALUES` : The input feature dimension.
- `M_VALUES` : The network width.
- `L_VALUES` : The network depth (referred to as H in our formulas).

For each configuration, the script performs the following steps :

1. **Initialization** : It generates random, normalized input data using `generate_data` and initializes the network weights according to a standard normal distribution via `init_network`.
2. **Forward Pass** : It calls `compute_features_and_derivatives` to perform a full forward pass, collecting all the necessary intermediate values : the feature maps ($x^{(p)}$) and the diagonal matrices of activation derivatives (σ'_p) for each layer.
3. **Kernel Computation** : An instance of the `Kernel3Empirical` class is created with the network weights and the results from the forward pass. The script then computes the entire $N \times N \times N$ tensor for $K^{(3)}$ by calling the `kernel3(i, j, k)` method for all unique triplets of indices, leveraging the tensor's symmetry.

28.2 Data Analysis and Connection to Theory

Once the empirical `k3_tensor` is computed, the script extracts quantitative metrics to analyze its properties.

- **Infinity Norm** : The script computes `inf_norm = M * np.max(np.abs(k3_tensor))`. The purpose of this analysis is to study the kernel's asymptotic behavior. By scaling the empirical result with a factor depending on the width M, we can check if the quantity converges to a finite, depth-dependent limit as $M \to \infty$. Our theoretical analysis predicted that the entries of $K^{(3)}$ scale as $O(H^2/\sqrt{M})$. Therefore, multiplying the maximum absolute value by $\sqrt{M}$ should result in a quantity that grows quadratically with depth H. The script's use of a scaling factor of M may be intended to test a different theoretical hypothesis or scaling regime.
- **Eigenvalue Analysis** : The script also performs a spectral analysis by examining the eigenvalues of 2D slices of the tensor (`k3_tensor[:, :, k]`). This provides deeper insight into the operator's properties than its maximum entry alone. The mean and standard deviation of these eigenvalues are computed across all slices to produce a statistical summary of the kernel's spectrum. These spectral properties are also scaled by the width M.

The data generated by this script (the computed norms and eigenvalue statistics for varying H and M) is saved and can be plotted to visually confirm or challenge the $O(H^2/\sqrt{M})$ scaling law derived theoretically. This

provides a crucial bridge between the analytical formulas of the NTH and their concrete implications for the behavior of finite-width neural networks.

29
DEEP DIVE INTO QUANTITATIVE METRICS

To bridge the gap between our theoretical analysis and the empirical results from the script, it's essential to understand precisely what the computed metrics represent and how they connect to the underlying theory.

29.1 Analysis of the Scaled Infinity Norm

The script computes `inf_norm = M * np.max(np.abs(k3_tensor))`. Let's break this down.

— **What is the Infinity Norm ?** The infinity norm of the $K^{(3)}$ tensor, denoted $\|K^{(3)}\|_\infty$, is simply the largest absolute value among all its entries :

$$\|K^{(3)}\|_\infty = \max_{\alpha,\beta,\gamma} |K^{(3)}_{\alpha\beta\gamma}|$$

This metric provides a simple way to quantify the overall magnitude of the kernel.

— **The Purpose of Scaling by Width** M : In the infinite-width limit ($M \to \infty$), the standard NTK theory shows that $K^{(3)}$ vanishes, which is why the kernel dynamics are ignored. Our analysis for finite-width networks aims to characterize precisely *how fast* it vanishes. The scaling factor is introduced to cancel out the width-dependent decay, thereby isolating the depth-dependent behavior.

— **Testing Theoretical Predictions** : Our detailed scaling analysis predicted that the dominant term in $K^{(3)}$ scales as $O(H^2/\sqrt{M})$. To empirically test this hypothesis, one should analyze the quantity $\sqrt{M}\cdot\|K^{(3)}\|_\infty$. If the theory is correct, this scaled value should converge to a limit that grows quadratically with depth (H^2) as M becomes large.

The experiment, by computing $M \cdot \|K^{(3)}\|_\infty$, is implicitly testing an alternative hypothesis where the kernel entries scale as $O(1/M)$. If our $O(1/\sqrt{M})$ theory is correct, the quantity computed in the script should not converge but instead grow like $\sqrt{M}$. By observing which of these scaled quantities (or another) stabilizes as M increases, the experiment can empirically determine the true scaling law, thus validating or falsifying different theoretical claims.

29.2 Spectral Analysis of Tensor Slices

The eigenvalue analysis provides a much deeper view into the structure of $K^{(3)}$ than the infinity norm alone. The dynamics of the NTK are given by the equation :

$$\frac{d}{dt}K^{(2)}_t = -\frac{1}{n}\sum_{\gamma=1}^{n}(f_t(x_\gamma) - y_\gamma)\cdot K^{(3)}_{t,\gamma}$$

where $K^{(3)}_{t,\gamma}$ is the $N \times N$ matrix slice of the tensor for a fixed index γ. The evolution of $K^{(2)}$ is thus a linear combination of these slice matrices.

— **Physical Meaning of Eigenvalues** : The eigenvalues of a slice matrix $K^{(3)}_\gamma$ characterize how an error at a single data point, x_γ, influences the geometry of the feature space (as captured by $K^{(2)}$). An eigenvector of this matrix represents a direction in the input space, and its corresponding eigenvalue represents the magnitude and sign of the change in that direction. A large positive eigenvalue means that

the kernel's value for pairs of points aligned with that eigenvector will increase, effectively pushing those representations further apart.

— **Mean and Standard Deviation** :
  — `mean_eigenvalues` : By averaging the eigenvalues of all slices, we obtain the "average" spectral signature of the $K^{(3)}$ operator. This tells us if there is a systematic, data-independent drift in the NTK. For instance, a consistently negative smallest eigenvalue across all slices would imply that gradient descent tends to contract the feature space along a specific direction, regardless of which data point has a large error.
  — `std_eigenvalues` : The standard deviation measures how much this spectral impact varies depending on the data point x_γ driving the update. A small standard deviation would imply that the kernel evolves in a very predictable, almost data-independent manner. Conversely, a large standard deviation suggests that the geometry of the kernel is being sculpted in a highly complex, data-dependent way, with each training example pulling the kernel in a different "direction".
— **Asymptotic Behavior** : As with the infinity norm, the eigenvalues are scaled by the width M to study their asymptotic behavior. The eigenvalues of a matrix with entries scaling as ϵ will also scale as ϵ. Therefore, if the entries of $K^{(3)}$ scale as $O(1/\sqrt{M})$, its eigenvalues should as well. The experiment provides the necessary data to check if the scaled spectrum converges to a stable, non-trivial limit as the width grows.

29.3 Explicit Indexed Formula for O_3

For clarity we restate the precise definition established in the proof above. Fix a parameter θ_μ with multi–index $\mu = (p, i, j)$ belonging to the weight matrix A_p. We encode the choice of μ by a third input x_3 and set

$$O_3(x_1, x_2, x_3) \;:=\; \partial_{\theta_\mu} K_\theta(x_1, x_2).$$

Because only the summand with $k = p$ depends on θ_μ, the derivative acts exclusively on that term. Specialising to multilayer perceptrons with ReLU activation one obtains the layer–wise expression

$$O_3(x_1, x_2, x_3) \;=\; \sigma^{-2}\, \partial_{A_{p,ij}} \big\langle x_p(x_1), x_p(x_2) \big\rangle, \tag{88}$$

where the indices (p, i, j) are implicitly determined by x_3. All contributions coming from deeper layers vanish since the second derivative of the ReLU satisfies $\phi'' \equiv 0$. Consequently O_3 is already of order $\mathcal{O}(n^{-1})$ and no further simplification is required.

29.4 Explicit Formulas for the Corrections

Having established the theoretical foundation, we now present the explicit formulas for the first-order corrections O_3 and O_4.

[First-order correction to the NTK] The first-order correction to the Neural Tangent Kernel admits the decomposition :

$$\Theta(x_1, x_2; t) = \Theta_{x_1, x_2; 0} + \Theta^{(1)}(x_1, x_2; t) + \mathcal{O}(n^{-2}) \tag{89}$$

where the correction term is given by :

$$\Theta^{(1)}(x_1, x_2; t) = -\int_0^t dt' \sum_{(x,y)\in D_{\text{tr}}} O_3^{(1)}(x_1, x_2, x; t') \left(f^{(0)}(x; t') - y \right) \tag{90}$$

[Eigendecomposition of corrections] Using the eigendecomposition of the initial kernel Θ_0, the correction can be expressed as :

$$\begin{aligned}\Theta^{(1)}(\vec{x};t) = &-\sum_i \frac{1}{\lambda_i}(O_3(\vec{x};0)\cdot\hat{e}_i)(\Delta f_0\cdot\hat{e}_i)\left[1-e^{-t\lambda_i}\right]\\ &+\sum_{ij}\frac{1}{\lambda_j}(\hat{e}_i^T O_4(\vec{x};0)\hat{e}_j)(\Delta f_0\cdot\hat{e}_i)(\Delta f_0\cdot\hat{e}_j)\\ &\times\left[\frac{1-e^{-t\lambda_i}}{\lambda_i}-\frac{1-e^{-t(\lambda_i+\lambda_j)}}{\lambda_i+\lambda_j}\right]\end{aligned} \tag{91}$$

where :

- $\hat{e}_i$ are the eigenvectors of the initial kernel Θ_0
- λ_i are the corresponding eigenvalues
- $\vec{x} = (x_1, x_2)$ represents the input pair
- $O_3(\vec{x};0)$ is a vector over the training dataset
- $O_4(\vec{x};0)$ is a square matrix over the training dataset
- $\Delta f_0 := f_0 - y$ is the initial prediction error vector

[Network evolution with corrections] The evolution of the network function including first-order corrections is governed by :

$$\frac{df(x;t)}{dt} = -\sum_{(x',y)\in D_{\text{tr}}}\left(\Theta(x,x';0)+\Theta^{(1)}(x,x';t)\right)(f(x';t)-y)+\mathcal{O}(n^{-2}) \tag{92}$$

with the solution :

$$f(t) = y + e^{-\Theta_0 t}\left(1-\int_0^t dt' e^{t'\Theta_0}\Theta^{(1)}(t')e^{-t'\Theta_0}\right)(f_0-y) \tag{93}$$

[Interpretation of the corrections] The formulas in thm :eigen$_c$*orrectionsrevealseveralimportantproperties* :

The first term involving O_3 represents the linear correction due to kernel evolution during training. This term scales as $\mathcal{O}(n^{-1})$ and captures how the finite width affects the kernel's constancy assumption.

The second term involving O_4 represents quadratic corrections that arise from the interaction between different eigenmodes of the initial kernel. The complex time dependence in the bracketed term shows how these corrections evolve differently from the linear term.

The exponential factors $e^{-t\lambda_i}$ demonstrate that corrections become more significant for eigenvalues close to zero, which corresponds to directions in function space where the network learns slowly.

29.5 Approches Alternatives et Concentration

(Explication de l'approche alternative pour les FCNN via les réseaux paraboliques de Terjek, permettant de se placer en régime de feature learning tout en conservant des propriétés du NTK. Discussion sur les bornes de concentration.)

30

ANALYSE D'ARCHITECTURES SPÉCIFIQUES

30.1 Réseaux à Mémoire Matricielle (MMNN) et Transformeurs

(Description des techniques de préconditionnement du NTK pour ces architectures. Discussion sur les régimes "feature" et "kernel", et sur les défis liés à l'optimisation. Présentation d'une conjecture sur la stabilité autour des minima globaux.)

31
MMNN NTK FOR 2 HIDDEN LAYERS

31.1 RECURSIVE

For a 2 hidden layer MMNN, we can write the recursive formulation of the NTK kernel. Let's recall the general recursive formula for layer ℓ :

suppose that you have a low rank hidden layer, with a width r

so the architecture is :

$$(x,y) \xrightarrow{\sigma W_1} h_1 \xrightarrow{A_1 \cdot + c_1} r \xrightarrow{\sigma W_2} h_2 \xrightarrow{A_2 \cdot + c_2} (x_2, y_2)$$
$$N \qquad (x_1, y_1) \text{ in } \mathbb{R}^r \qquad N$$

And here N grow to infinity, and r is fixed.

Then the NTK random kernel is :

$$\Theta^{(\ell)}(x,x') = \Theta^{(\ell-1)}(x,x') \cdot \left(\sigma_A^2 \dot{\Sigma}^{(\ell)}(x,x')\right) + \Sigma^{(\ell)}(x,x') + 1 \tag{94}$$

For $\ell = 2$, starting from $\Theta^{(0)} = 0$, we have :

$$\begin{aligned}
\Theta^{(1)} &= \mathbb{E}_{w,b}\left[\sigma(w^\top x + \beta b)\sigma(w^\top x' + \beta b)\right] + 1 \\
\Theta^{(2)} &= \Theta^{(1)} \cdot \left(\sigma_A^2 \mathbb{E}_{w,b}\left[\dot{\sigma}(w^\top h^{(1)}(x) + \beta b)\dot{\sigma}(w^\top h^{(1)}(x') + \beta b) w^\top w\right]\right) \\
&\quad + \mathbb{E}_{w,b}\left[\sigma(w^\top h^{(1)}(x) + \beta b)\sigma(w^\top h^{(1)}(x') + \beta b)\right] + 1
\end{aligned}$$

Here, $\Theta^{(1)}$ is a deterministic NNGP kernel (Gaussian Process) since it depends only on the input x, while $\Theta^{(2)}$ is a random field (Deep Gaussian Process) that depends on the realization of the first layer's output $h^{(1)}$. When $\beta = 0$, these expectations simplify by removing the bias terms b. This non-Gaussian nature of the kernels for $\ell > 1$ is a key distinguishing feature of MMNNs compared to standard neural networks.

When $\beta = 0$, the equations simplify to :

Main focus : NTK kernels with β

$$\Theta^{(1)} = \mathbb{E}_w\left[\sigma(w^\top x)\sigma(w^\top x')\right] + 1$$
$$\Theta^{(2)} = \Theta^{(1)} \cdot \left(\sigma_A^2 \mathbb{E}_w\left[\dot\sigma(w^\top h^{(1)}(x))\dot\sigma(w^\top h^{(1)}(x'))w^\top w\right]\right)$$
$$+ \mathbb{E}_w\left[\sigma(w^\top h^{(1)}(x))\sigma(w^\top h^{(1)}(x'))\right] + 1$$

Note that for stable signal propagation at the Edge of Chaos (EOC), we require for 1 rank hidden layer :

$$\sigma_A^2 \cdot \mathrm{Tr}(\Sigma_w) = 2$$

Then for a rank r

$$\sigma_A^2 \cdot \mathrm{Tr}(\Sigma_w) = \frac{2}{r}$$

And we assume at least isotropic gaussian for the weights without loss of generality and to ensure fast computations.

This condition ensures the signal neither vanishes nor explodes as it propagates through the network layers. This also means we need to normalize by $\sqrt{r}$ the output of the first MMNN layer. We call x_1 and y_1 the outputs of the first layer unnormalized.

We recall also that $\Sigma^{(1)}$ is the arccosine kernel, which has a closed form in function of the correlation ρ between x and x' :

$$\Sigma^{(1)}(\rho) = \frac{1}{\pi}\left(\sqrt{1-\rho^2} + \rho(1 - \arccos(\rho))\right)$$

We recall also that $\dot\Theta^{(1)}(\rho) = \mathbb{E}_w\left[w^\top w \dot\sigma(w^\top x)\dot\sigma(w^\top x')\right]$. We can compute this exactly when w is isotropic gaussian with variance 1 :

In fact a simple computation takes the expectation of the product of the two derivatives of the ReLU, by independence of $\|w\|^2$ and $\mathbb{1}_{w^\top x>0} \cdot \mathbb{1}_{w^\top x'>0}$:

$$\begin{aligned}\dot\Theta^{(1)}(\rho) &= \mathbb{E}_w\left[w^\top w \dot\sigma(w^\top x)\dot\sigma(w^\top x')\right] \\ &= \mathbb{E}_w\left[w^\top w \cdot \mathbb{1}_{w^\top x>0} \cdot \mathbb{1}_{w^\top x'>0}\right] \\ &= \mathbb{E}\left[\|w\|^2\right] \cdot \mathbb{P}(w^\top x > 0, w^\top x' > 0) \\ &= \mathrm{Tr}(\Sigma_w)\left(\frac{1}{2} - \frac{\arccos(\rho)}{2\pi}\right)\end{aligned}$$

It is just a reformulation of the fact that we integrate over a cone of angle $\arccos(\rho)$ a radial function.

We then write $\Theta^{(2)}$ as a function of ρ and ρ_1 :

$$\Theta^{(2)}(\rho,\rho_1) = \Theta^{(1)}(\rho) \cdot \sigma_A^2\,\mathrm{Tr}(\Sigma_w)\left(\frac{1}{2} - \frac{\arccos(\rho)}{2\pi}\right) + \frac{1}{r} \cdot \Sigma^{(1)}(\rho_1)\|x_1\|\|y_1\| + 1$$

$$\text{with} \quad \Sigma^{(1)}(\rho) = \frac{1}{\pi}\left(\sqrt{1-\rho^2} + \rho(1 - \arccos(\rho))\right)$$

where ρ_1 is the correlation between the outputs of the first layer, that is a random variable and ρ is the correlation between the inputs.

Note that this multiplicative value is between 0 and 1, that is we can compute a lot of stuff after concatenating layers and getting some exponential scaling laws. We emphasize the fact we multiply by $\|x_1\|\|y_1\|$ which is a random variable, but as we will see, it is independent of the correlation ρ_1.

31.2 Expression for random variables ρ_1, $\|x_1\|^2$, $\|y_1\|^2$

The output of the 1st layer is a gaussian process of dimension r, where each coordinate is a gaussian process with covariance matrix :

$$\Sigma_i^{(1)} = \mathbb{E}_w \left[\sigma(w^\top x)\sigma(w^\top x')\right]$$

and the coordinates are ρ-correlated. If we stack horizontally the outputs x_1 and y_1 in a vector of dimension $2r$, the full $2r \times 2r$ covariance matrix is therefore :

$$\Sigma^{(1)} = \begin{pmatrix} I & \rho I \\ \rho I & I \end{pmatrix}$$

It's a kronecker product of r matrices I and 2d ρ correlated gaussian variables.

it happens, by a magic, that we can compute exactly the joint distribution of the random variable $(\rho_1, \|x_1\|^2, \|y_1\|^2)$:

$$\rho_1 = \frac{\sum_{i=1}^r x_i y_i}{\sqrt{\sum_{i=1}^r x_i^2}\sqrt{\sum_{i=1}^r y_i^2}}$$

The squared norms x_1^2 and y_1^2 follow a bivariate Kibble distribution and their cosine distance follows an independent Fisher like distribution. The Kibble distribution is defined for $u, v \geq 0$ and $\rho \in [-1, 1]$, while the Fisher distribution is defined for $\theta \in [-1, 1]$ and $\rho \in [-1, 1]$ with $r > 2$.

$$\begin{aligned}(\|x_1\|^2, \|y_1\|^2) &\sim \text{Kibble}(r, \rho) \\ &= \frac{(uv)^{\frac{r/2-1}{2}} e^{-\frac{u+v}{2(1-\rho^2)}}}{\Gamma(r/2)(2(1-\rho^2))^{r/2+1}\rho^{\frac{r/2-1}{2}}} \cdot I_{r/2-1}\left(\frac{\rho\sqrt{uv}}{1-\rho^2}\right)\end{aligned}$$

$$\begin{aligned}\rho_1 &\sim \text{Fisher}(\rho, r) \\ &= \frac{(r-2)\Gamma(r-1)(1-\rho^2)^{\frac{r-1}{2}}(1-\theta^2)^{\frac{r-4}{2}}}{\sqrt{2\pi}\Gamma(r-\frac{1}{2})(1-\rho\theta)^{r-\frac{3}{2}}} \times {}_2F_1\left(\frac{1}{2}, \frac{1}{2}; r-\frac{1}{2}; \frac{1+\rho\theta}{2}\right)\end{aligned}$$

where $u = \|x_1\|^2$, $v = \|y_1\|^2$, $\theta = \cos(x_1, y_1)$, and I_α is the modified Bessel function of the first kind.

and the correlation of x,y :

where x_{1i} and y_{1i} are ρ-correlated gaussian variables.

We ensure that the reader has understood that the correlation of x,y is independent of the squared norms of x,y !

31.2.1 • Final result

We can then state the final result : For 2 entries x,y in $\mathbb{R}^d$ and a low rank r hidden layer, the NTK for a 2 hidden layer MMNN is :

$$\Theta^{(2)}(x, y) = \Theta^{(1)}(\rho) \cdot \left(1 - \frac{\arccos(\rho_1)}{\pi}\right)\frac{1}{r} + \frac{1}{r}\Sigma^{(1)}(\rho_1)\|x_1\|\|y_1\| + 1$$

$$\text{with} \quad \Sigma^{(1)}(\rho) = \frac{1}{\pi}\left(\sqrt{1-\rho^2} + \rho(1 - \arccos(\rho))\right)$$

where the random variables follow :

- $\rho_1 \sim \text{Fisher}(\rho, r)$ is independent of the norms
- $(\|x_1\|^2, \|y_1\|^2) \sim \text{Kibble}(r, \rho)$ are chi-squared variables with r degrees of freedom from correlated gaussian variables.

31.3 Intuitions sur les Performances

(Discussion finale basée sur les résultats théoriques et expérimentaux pour expliquer les performances observées pour les MMNN, les transformeurs et les ResNets.)

32

RÉSULTATS EXPÉRIMENTAUX ET APPLICATIONS

32.1 Validation des Lois de Scaling et des Corrections

(Présentation des expériences numériques validant les lois de scaling pour les corrections à largeur finie dans les FCNN.)

32.2 Analyse Hessienne et Dynamique

(Présentation des expériences sur l'approche hessienne du NTK et l'étude des termes de type Lyapunov.)

32.3 Applications en SciML et Discussion

(Discussion sur l'application des résultats en Scientific Machine Learning. Vérification des hypothèses théoriques sur les grands modèles et le scaling des paramètres. Lien avec la physique statistique et la convergence vers l'équilibre thermique.)

ANNEXES

Figure 2 – Aperçu de l'interface mise à disposition en source ouverte pour l'extraction de connaissances

A
FISHER, KIBBLE DISTRIBUTIONS AND THE FAKE "CURSE OF DIMENSIONALITY"

A.1 Distribution of the Correlation Coefficient r

Fisher derived the exact law of r in 1915, representing the cosine between Gaussian vectors of dimension r with correlation ρ :

$$p(r|\rho, r) = \frac{(r-2)\Gamma(r-1)(1-\rho^2)^{\frac{r-1}{2}}(1-r^2)^{\frac{r-4}{2}}}{\sqrt{2\pi}\Gamma(r-\frac{1}{2})(1-\rho r)^{r-\frac{3}{2}}} \times {}_2F_1\left(\frac{1}{2}, \frac{1}{2}; r-\frac{1}{2}; \frac{1+\rho r}{2}\right)$$

Where Γ is the gamma function and ${}_2F_1$ is the hypergeometric function.

For large r, Fisher's z-transform gives $z = \operatorname{arctanh}(r) \sim \mathcal{N}\left(\operatorname{arctanh}(\rho) + \frac{\rho}{2(r-1)}, \frac{1}{r-3}\right)$. The variance of r decreases as $O(1/r)$. When $\rho \sim O(1/\sqrt{r})$, geometric noise masks correlation.

A.2 The Kibble Distribution

The joint law of correlated chi-squared variables $U = \|X\|^2$ and $V = \|Y\|^2$ follows Kibble's 1941 distribution :

$$f(u,v) = \frac{(uv)^{\frac{r/2-1}{2}} e^{-\frac{u+v}{2(1-\rho^2)}}}{\Gamma(r/2)(2(1-\rho^2))^{r/2+1}\rho^{\frac{r/2-1}{2}}} \cdot I_{r/2-1}\left(\frac{\rho\sqrt{uv}}{1-\rho^2}\right)$$

Where Γ is the gamma function and I_α is the modified Bessel function of the first kind.

Mean $\mathbb{E}[U] = r$ and variance $\operatorname{Var}(U) = 2r$ grow linearly. U/r converges to $\mathcal{N}(1, 2/r)$ with polynomial concentration.

B
PRICE'S THEOREM AND DERIVATIONS

Price's theorem was a central tool in our analysis.

- Statement of price's theorem

Suppose that X_1 and X_2 forms a gaussian vector with joint distribution $f_{X_1,X_2}(x_1, x_2)$. Then, for any function $g(x_1, x_2)$, we have :

$$\mathbb{E}[g(X_1, X_2)] = \int_0^\rho \mathbb{E}\left[\frac{\partial^2 g}{\partial x_1 \partial x_2}(X_1, X_2)\right] d\rho + \text{Constant}$$

where the constant is the value of the expectation when $\rho = 0$.

• Calculation of I_{12} and I_{21} (First-Order Moments)

For I_{12}, we need to calculate :

$$I_{12} = \int_a^\infty \int_b^\infty z_2 \phi_2(z_1, z_2; \rho)\, dz_2\, dz_1$$

Step 1 : Inner integral We first focus on the inner integral with respect to z_2. The exact formula for the conditional density is :

$$\phi_2(x, y; \rho) = \frac{1}{\sqrt{1-\rho^2}} \phi_1(y) \phi_1\left(\frac{x - \rho y}{\sqrt{1-\rho^2}}\right)$$

Applying this formula with $x = z_1$ and $y = z_2$:

$$\phi_2(z_1, z_2; \rho) = \frac{1}{\sqrt{1-\rho^2}} \phi_1(z_2) \phi_1\left(\frac{z_1 - \rho z_2}{\sqrt{1-\rho^2}}\right)$$

The inner integral becomes :

$$\int_b^\infty z_2 \frac{1}{\sqrt{1-\rho^2}} \phi_1(z_2) \phi_1\left(\frac{z_1 - \rho z_2}{\sqrt{1-\rho^2}}\right) dz_2$$

Step 2 : Change of variable Let $u = \frac{z_1 - \rho z_2}{\sqrt{1-\rho^2}}$. Then :

$$z_2 = \frac{z_1 - u\sqrt{1-\rho^2}}{\rho}$$
$$dz_2 = -\frac{\sqrt{1-\rho^2}}{\rho}\, du$$

The lower bound becomes $u_{max} = \frac{z_1 - \rho b}{\sqrt{1-\rho^2}}$. The integral transforms into :

$$\frac{1}{\sqrt{1-\rho^2}} \int_{-\infty}^{\frac{z_1-\rho b}{\sqrt{1-\rho^2}}} \left(\frac{z_1 - u\sqrt{1-\rho^2}}{\rho}\right) \phi_1\left(\frac{z_1 - u\sqrt{1-\rho^2}}{\rho}\right) \phi_1(u) \left(-\frac{\sqrt{1-\rho^2}}{\rho}\right) du$$

Step 3 : Simplification Grouping terms and using properties of ϕ_1 :

$$-\frac{1}{\rho^2} \int_{-\infty}^{\frac{z_1-\rho b}{\sqrt{1-\rho^2}}} (z_1 - u\sqrt{1-\rho^2}) \phi_1(u) \phi_1\left(\frac{z_1 - u\sqrt{1-\rho^2}}{\rho}\right) du$$

Step 4 : Using the conditional density formula The exact formula is :

$$\phi_2(x, y; \rho) = \frac{1}{\sqrt{1-\rho^2}} \phi_1(x) \phi_1\left(\frac{y - \rho x}{\sqrt{1-\rho^2}}\right)$$

Applying it correctly to our case with $x = z_1$ and $y = z_2$:

$$I_{12} = \phi_1(b) \Phi_1\left(\frac{a + \rho b}{\sqrt{1-\rho^2}}\right) + \rho \phi_1(a) \Phi_1\left(-\frac{b + \rho a}{\sqrt{1-\rho^2}}\right)$$

This final formula for I_{12} is one of the key terms in calculating the expectation of the product of ReLUs.

- Calculation of I_{22} (Cross-Moment)

The calculation of I_{22} is the most complex part. The final result is :

$$I_{22} = \int_0^\rho I_{11} + \phi_2(a, b; \rho)$$

The proof of this identity was a central point of the discussion. The most rigorous method consists of showing the equality for $\rho = 0$, and then the equality of the derivatives with respect to ρ for both sides of the equation.

- Final Formula

By assembling all the terms, we obtain the final algebraic formula :

$$\mathbb{E}[\dots] = (\mu_1\mu_2 + \rho\sigma_1\sigma_2)I_{11} + \mu_1\sigma_2 I_{12} + \mu_2\sigma_1 I_{21} + \sigma_1\sigma_2\phi_2(a, b; \rho)$$

B.1 Computing ReLU Product Expectations

When analyzing the Neural Tangent Kernel (NTK) of a single-layer network, we need to compute expectations of the form :

$$\mathbb{E}[\text{ReLU}(X_1)\text{ReLU}(X_2)]$$

where (X_1, X_2) follows a bivariate normal distribution with parameters $(\mu_1, \sigma_1, \mu_2, \sigma_2, \rho)$.

For deterministic input vectors x_1 and x_2, we define the following quantities :

- $\sigma_1 = \|x_1\|$ (where $\|\cdot\|$ denotes Euclidean norm)
- $\sigma_2 = \|x_2\|$
- $\rho = \cos(x_1, x_2) = \frac{\langle x_1, x_2 \rangle}{\|x_1\|\|x_2\|}$

The network outputs follow a Gaussian process characterized by the covariance matrix :

$$\Sigma = \begin{pmatrix} \sigma_1^2 & \rho\sigma_1\sigma_2 \\ \rho\sigma_1\sigma_2 & \sigma_2^2 \end{pmatrix}$$

$\sigma_1^2\rho\sigma_1\sigma_2$
$\rho\sigma_1\sigma_2\sigma_2^2$

B.2 Decomposition

We recall the expression of mu The approach is to standardize the variables ($Z_i = (X_i - \mu_i)/\sigma_i$) and decompose the integral into four terms :

$$\mathbb{E}[\dots] = \sigma_1\sigma_2 I_{22} + \mu_1\sigma_2 I_{12} + \mu_2\sigma_1 I_{21} + \mu_1\mu_2 I_{11}$$

where the integrals I_{ij} are defined over the domain $z_1 > a$ and $z_2 > b$, with $a = -\mu_1/\sigma_1$ and $b = -\mu_2/\sigma_2$.

- $I_{11} = \int_a^\infty \int_b^\infty \phi_2(z_1, z_2; \rho)\, dz_2\, dz_1$
- $I_{21} = \int_a^\infty \int_b^\infty z_1\phi_2(z_1, z_2; \rho)\, dz_2\, dz_1$
- $I_{12} = \int_a^\infty \int_b^\infty z_2\phi_2(z_1, z_2; \rho)\, dz_2\, dz_1$
- $I_{22} = \int_a^\infty \int_b^\infty z_1 z_2\phi_2(z_1, z_2; \rho)\, dz_2\, dz_1$

B.3 Calculation of the Components

• Calculation of I_{11} (Probability)

I_{11} is the probability $P(Z_1 > a, Z_2 > b)$. It is calculated via the cumulative distribution function (CDF) of the standard bivariate normal distribution, Φ_2.

$$I_{11} = P(Z_1 > a, Z_2 > b) = P(Z_1 < -a, Z_2 < -b) = \Phi_2(-a, -b; \rho) = \Phi_2\left(\frac{\mu_1}{\sigma_1}, \frac{\mu_2}{\sigma_2}; \rho\right)$$

C
UNSUCCESSFUL ATTEMPTS TO CALCULATE I_{22}

Several approaches to calculate I_{22} proved to be dead ends.

C.1 Via the Identity on the Derivative of ϕ_2

Let's first prove the identity for the partial derivatives of the 2D normal PDF ϕ_2. For a bivariate normal distribution with correlation ρ :

$$\frac{\partial \phi_2}{\partial z_1} = -\frac{z_1 - \rho z_2}{1-\rho^2}\phi_2(z_1, z_2; \rho)$$
$$\frac{\partial \phi_2}{\partial z_2} = -\frac{z_2 - \rho z_1}{1-\rho^2}\phi_2(z_1, z_2; \rho)$$

Using these identities, we can write :

$$z_1\phi_2 = -\rho z_2\phi_2 - (1-\rho^2)\frac{\partial \phi_2}{\partial z_1}$$

Integrating from a to ∞ with respect to z_1 :

$$\int_a^\infty z_1\phi_2\, dz_1 = -\rho z_2 \int_a^\infty \phi_2\, dz_1 + (1-\rho^2)\phi_2(a, z_2; \rho)$$

Attempting to use this for I_{22}, we integrate $z_1 z_2 \phi_2$ over the region $[a, \infty) \times [b, \infty)$:

$$I_{22} = \int_b^\infty z_2 \left(\int_a^\infty z_1\phi_2\, dz_1 \right) dz_2$$

Substituting the identity :

$$I_{22} = -\rho \int_b^\infty z_2^2 \left(\int_a^\infty \phi_2\, dz_1 \right) dz_2 + (1-\rho^2) \int_b^\infty z_2\phi_2(a, z_2; \rho)\, dz_2$$

This approach becomes intractable because : 1. The first term involves a higher-order moment z_2^2 2. The second term requires evaluating another complex integral 3. Each attempt to simplify leads to terms of equal or greater complexity

Therefore, while the partial derivative identities are mathematically elegant, they do not provide a practical path to computing I_{22} when beta is non 0.

D
SPECIAL CASE : ZERO MEANS ($\mu_1 = \mu_2 = 0$)

In this case, the "from scratch" derivation leads to an elegant formula.

$$\mathbb{E}[\measuredangle(X_1)\ \measuredangle(X_2)] = \sigma_1\sigma_2 \int_0^\infty \int_0^\infty z_1 z_2 \phi_2(z_1, z_2; \rho)\, dz_1\, dz_2$$

The problem reduces to calculating $I_{22}(0,0;\rho) = \mathbb{E}[\measuredangle(Z_1)\ \measuredangle(Z_2)]$. The cleanest derivation uses Price's theorem :

1. $\frac{\partial \mathbb{E}[g]}{\partial \rho} = \mathbb{E}\left[\frac{\partial^2 g}{\partial z_1 \partial z_2}\right] = \mathbb{E}[\mathbb{1}(z_1 > 0, z_2 > 0)] = P(Z_1 > 0, Z_2 > 0) = \frac{1}{2\pi}\arccos(-\rho)$.
2. The integration constant is $\mathbb{E}[g]_{\rho=0} = \mathbb{E}[\measuredangle(Z_1)]\mathbb{E}[\measuredangle(Z_2)] = \left(\frac{1}{\sqrt{2\pi}}\right)^2 = \frac{1}{2\pi}$.
3. By integrating $\frac{\partial \mathbb{E}[g]}{\partial \rho}$ from 0 to ρ :

$$\begin{aligned}\mathbb{E}[g](\rho) - \mathbb{E}[g](0) &= \int_0^\rho \frac{1}{2\pi}\arccos(-t)\, dt \\ &= \frac{1}{2\pi}\left[t\arccos(-t) + \sqrt{1-t^2}\right]_0^\rho \\ &= \frac{1}{2\pi}(\rho\arccos(-\rho) + \sqrt{1-\rho^2} - 1)\end{aligned}$$

4. By adding the constant $\mathbb{E}[g](0) = 1/(2\pi)$, we get :

$$\mathbb{E}[\measuredangle(Z_1)\ \measuredangle(Z_2)] = \frac{1}{2\pi}(\sqrt{1-\rho^2} + \rho\arccos(-\rho))$$

The final formula for variances σ_1^2, σ_2^2 is :

$$\mathbb{E}[\measuredangle(X_1)\ \measuredangle(X_2)] = \frac{\sigma_1\sigma_2}{2\pi}(\sqrt{1-\rho^2} + \rho\arccos(-\rho))$$

D.1 Explicit Formulas for Σ and $\dot{\Sigma}$

For layer ℓ, the general formulas are :

$$\begin{aligned}\Sigma^{(\ell)}(x,x') &= \mathbb{E}_{w,b}\left[\sigma(w^\top h^{(\ell-1)}(x) + \beta b)\sigma(w^\top h^{(\ell-1)}(x') + \beta b)\right] \\ \dot{\Sigma}^{(\ell)}(x,x') &= \mathbb{E}_{w,b}\left[\dot{\sigma}(w^\top h^{(\ell-1)}(x) + \beta b)\dot{\sigma}(w^\top h^{(\ell-1)}(x') + \beta b)w^\top w\right]\end{aligned}$$

When $\beta = 0$, these simplify to :

$$\begin{aligned}\Sigma^{(\ell)}(x,x') &= \mathbb{E}_{w}\left[\sigma(w^\top h^{(\ell-1)}(x))\sigma(w^\top h^{(\ell-1)}(x'))\right] \\ \dot{\Sigma}^{(\ell)}(x,x') &= \mathbb{E}_{w}\left[\dot{\sigma}(w^\top h^{(\ell-1)}(x))\dot{\sigma}(w^\top h^{(\ell-1)}(x'))w^\top w\right]\end{aligned}$$

Note that for $\ell > 1$, both kernels become random fields due to the randomness in $h^{(\ell-1)}$.

D.2 Differential and Integral Form

The theorem relates the derivative of the expectation of a function g with respect to the covariance σ_{12} to the expectation of the second derivative of g. For $g = \_\!\!/(X_1)\ \_\!\!/(X_2)$, we have $\frac{\partial^2 g}{\partial x_1 \partial x_2} = \nVdash(X_1 > 0, X_2 > 0)$.

$$\frac{\partial \mathbb{E}[\_\!\!/(X_1)\ \_\!\!/(X_2)]}{\partial \sigma_{12}} = \mathbb{E}[\nVdash(X_1 > 0, X_2 > 0)] = I_{11}$$

This leads to the integral formulation :

$$\mathbb{E}[\_\!\!/(X_1)\ \_\!\!/(X_2)] = \int I_{11}(\sigma_{12})\, d\sigma_{12} + \text{Constant}$$

D.3 Calculating the Constant (Independent Case $\rho = 0$)

The constant is the value of the expectation when $\rho = 0$, i.e., when X_1 and X_2 are independent.

$$C = \mathbb{E}[\_\!\!/(X_1)] \cdot \mathbb{E}[\_\!\!/(X_2)]$$

The expectation of a single ReLU is :

$$\mathbb{E}[\_\!\!/(X)] = \sigma \phi_1\left(\frac{\mu}{\sigma}\right) + \mu \Phi_1\left(\frac{\mu}{\sigma}\right)$$

The constant is therefore the product of two such terms :

$$C = \left[\sigma_1 \phi_1\left(\frac{\mu_1}{\sigma_1}\right) + \mu_1 \Phi_1\left(\frac{\mu_1}{\sigma_1}\right)\right] \times \left[\sigma_2 \phi_1\left(\frac{\mu_2}{\sigma_2}\right) + \mu_2 \Phi_1\left(\frac{\mu_2}{\sigma_2}\right)\right]$$

This is the implementation of the constant in code, this is fairly long for non zeros beta, that's why we focus on the hypothesis with beta = 0. This in order to make experiments and proofs that MMNNs are superior to FCNNs for 2 hidden layer networks.

D.4 Exact moments and correlation for Kibble variables

Let $(X_i, Y_i)_{i=1}^r$ be i.i.d. Gaussian pairs with zero mean, unit variances and correlation ρ, and define

$$U = \sum_{i=1}^{r} X_i^2, \qquad V = \sum_{i=1}^{r} Y_i^2.$$

Then (U, V) follows Kibble's bivariate chi-square law with r degrees of freedom and correlation ρ. By Isserlis' (Wick's) theorem (or by expanding $\mathbb{E}[(X_i - \rho Y_i)^n (Y_i)^m]$ recursively) :

$$\mathbb{E}[X_i^2] = 1, \quad \mathrm{Var}(X_i^2) = 2, \quad \mathbb{E}[X_i^2 Y_i^2] = 1 + 2\rho^2, \quad (X_i^2, Y_i^2) = 2\rho^2.$$

Using independence across i :

$$\mathbb{E}[U] = r, \quad \mathrm{Var}(U) = 2r, \qquad \mathbb{E}[V] = r, \quad \mathrm{Var}(V) = 2r,$$

$$(U,V) = \sum_{i=1}^{r} (X_i^2, Y_i^2) = 2r\,\rho^2,$$

$$(U,V) = \frac{(U,V)}{\sqrt{\mathrm{Var}(U)\,\mathrm{Var}(V)}} = \frac{2r\rho^2}{\sqrt{(2r)(2r)}} = \rho^2,$$

$$\mathbb{E}[UV] = \mathbb{E}[U]\mathbb{E}[V] + (U,V) = r^2 + 2r\,\rho^2.$$

D.5 Product of Square Roots of Kibble Variables

Let $S = \sqrt{U}\sqrt{V} = \|X\|\|Y\|$. Due to correlation between U and V, we cannot directly multiply expectations. Instead, using Cauchy-Schwarz inequality :

$$\mathbb{E}[S^2] = \mathbb{E}[UV] = \mathbb{E}[U]\mathbb{E}[V] + (U,V) = r^2 + 2r\rho^2$$

Therefore :

$$\mathbb{E}[S] \leq \sqrt{\mathbb{E}[S^2]} = r\sqrt{1 + \frac{2\rho^2}{r}}$$

The variance can be estimated similarly by expanding $(S - \mathbb{E}[S])^2$ and using correlation terms :

$$\mathrm{Var}(S) \approx \frac{r}{2}(1+\rho^2) + O(1)$$

This suggests S/r approaches a distribution with mean $O(\sqrt{1 + \frac{2\rho^2}{r}})$ and variance of order $O(1/r)$.

D.6 Independence Properties and Asymptotic Behavior

The orientation variable r is independent of magnitudes (U,V). For the normalized product S/r, we have :

$$\frac{S}{r} = \frac{\|X\|\|Y\|}{r} \xrightarrow{r\to\infty} 1$$

This convergence occurs with fluctuations of order $O(1/\sqrt{r})$, showing that in high dimensions, the normalized product concentrates around 1 regardless of the correlation ρ.

E

EXPÉRIENCES NUMÉRIQUES

(paragraphe)